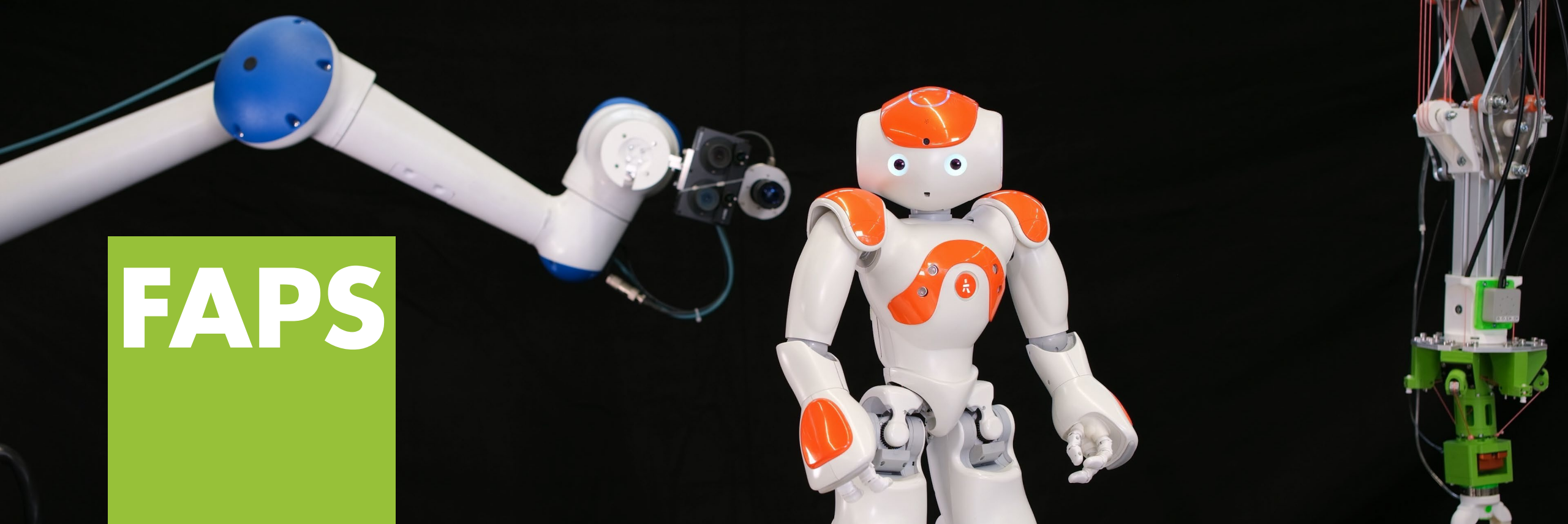




Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

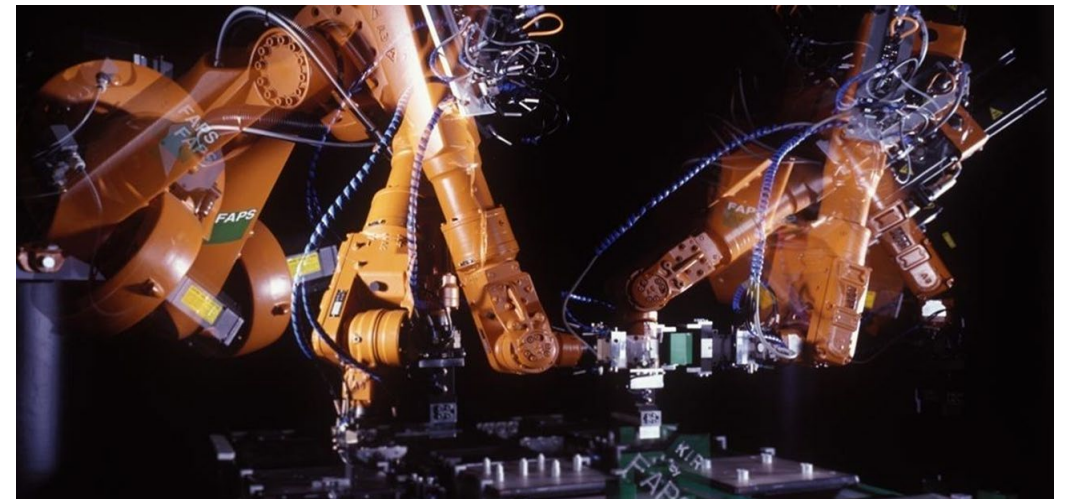
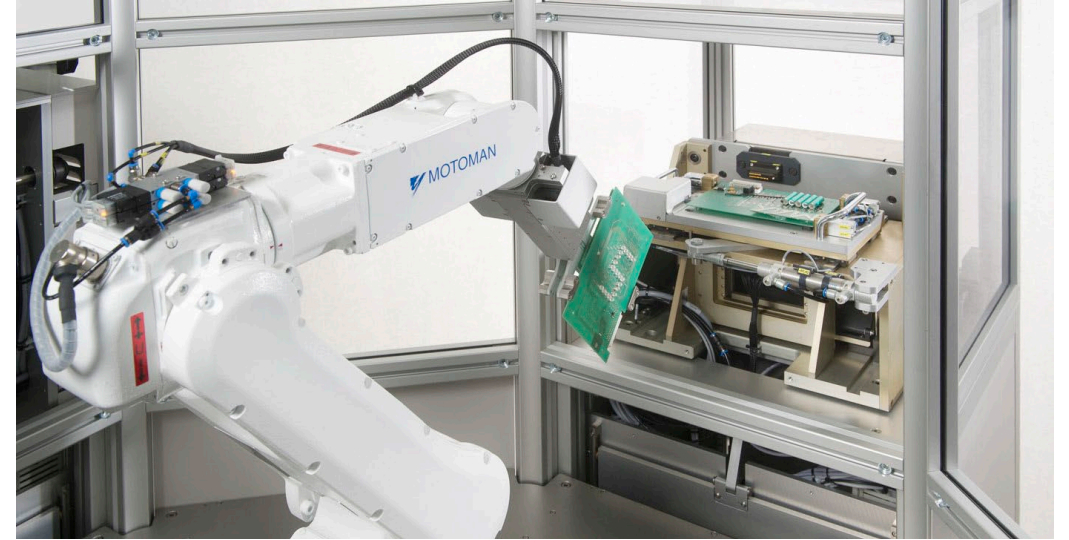
Grundlagen der Robotik (GdR)

VE07 – ROS - Erweiterte Konzepte und Tools

Im Fach Grundlagen der Robotik werden die fachspezifischen Grundlagen zur Konzeption, Implementierung und Realisierung von Robotersystemen vermittelt.

VE	Thema
----	-------

- | | |
|----|--|
| 1 | Einführung und Grundlagen der Industrierobotik & Neuartige Konzepte und Kinematiken in der Robotik |
| 2 | Steuerung - Sensorik - Aktorik |
| 3 | Roboterkinematik |
| 4 | Roboterdynamik |
| 5 | Bahnplanung und Regelung |
| 6 | Grundlagen des Robot Operating System (ROS) |
| 7 | ROS - Erweiterte Konzepte und Tools |
| 8 | Simulation und Planung |
| 9 | Rechnersehen |
| 10 | Maschinelles Lernen |
| 11 | Seilrobotik |
| 12 | Service-, Pflege- und Medizinrobotik |

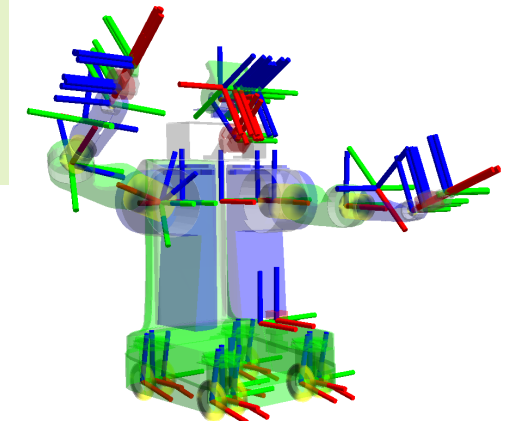
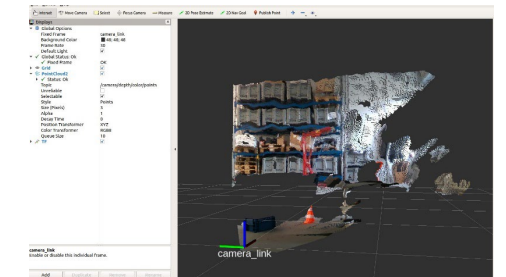


In der Vorlesungseinheit „ROS – Erweiterte Konzepte und Tools“ werden erweiterte Konzepte und Mechanismen des ROS vermittelt sowie Werkzeuge vorgestellt.

Das **Robot Operating System** stellt unterschiedliche **Kommunikationsmöglichkeiten**, **Algorithmen** sowie **Werkzeuge** zur **Steuerung** und zur **Entwicklung** von **Software** für **Roboter** zur Verfügung.

Nach dem Besuch dieser Vorlesungseinheit sollten Sie in der Lage sein:

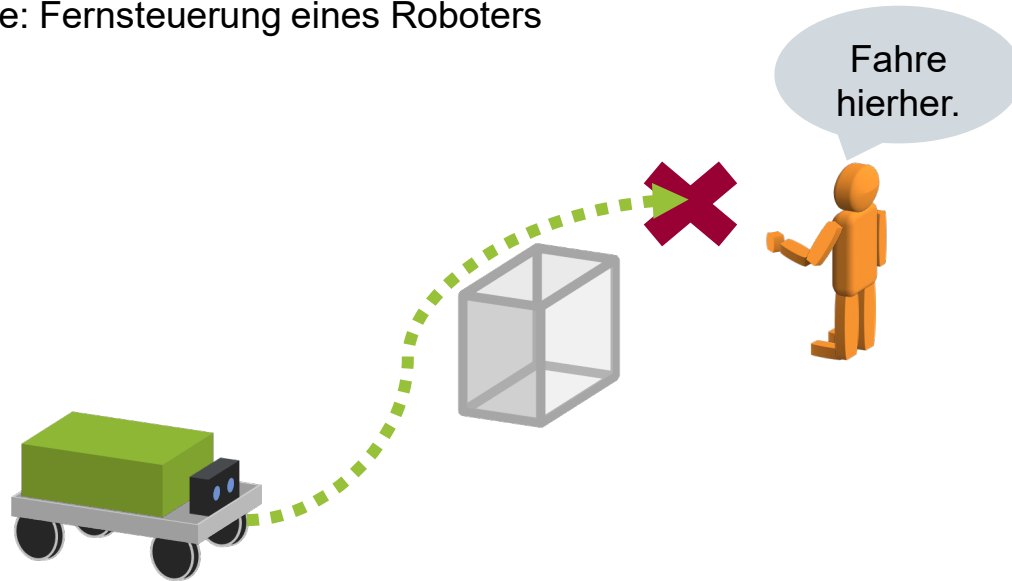
- das Prinzip der **Kommunikation** mithilfe von **ROS Actions** zu beschreiben,
- die **unterschiedlichen Kommunikationsarten** des ROS zu erläutern und zu vergleichen,
- unterschiedliche **Tools** des ROS einzuordnen und zu nutzen,
- den **Einsatz** und **Nutzen** von **Koordinatensystemen** innerhalb des ROS zu beschreiben,
- **Software** zur **Steuerung** eines **Robotersystem** mithilfe des ROS zu konzeptionieren und
- **Software** basierend auf dem **ROS** implementieren zu können.



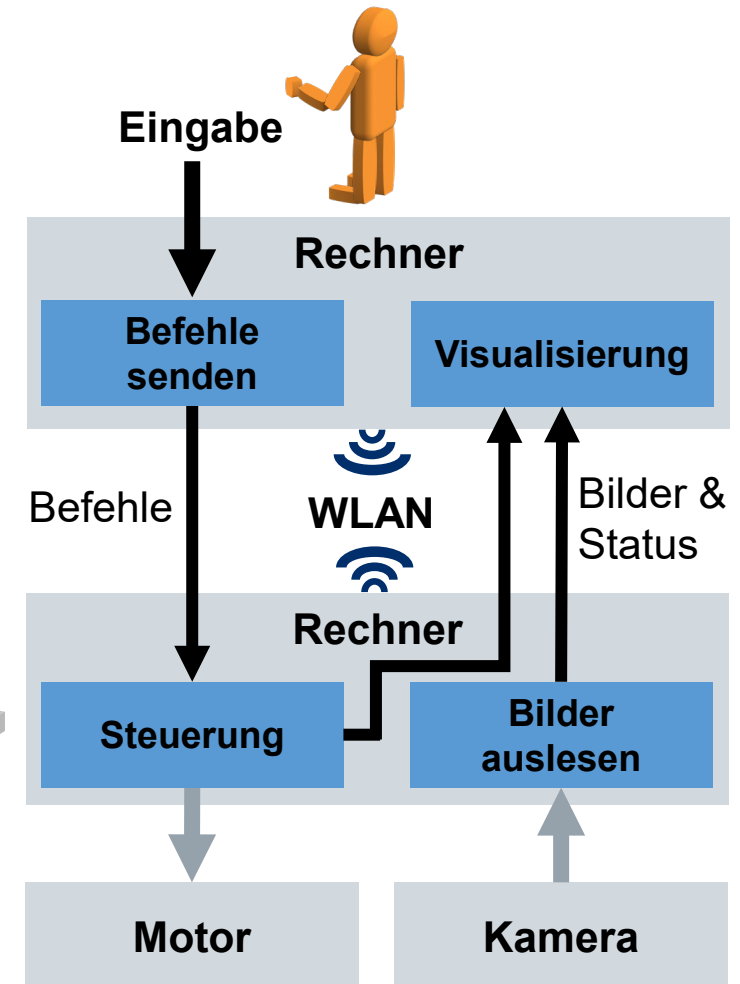
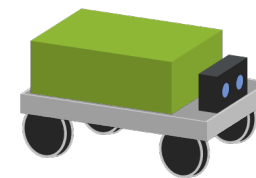
<https://docs.ros.org/en/humble/Concepts/About-Tf2.html>

In dieser Vorlesungseinheit werden die grundlegenden Funktionen des ROS anhand eines einfachen Beispiels vorgestellt.

■ Aufgabe: Fernsteuerung eines Roboters

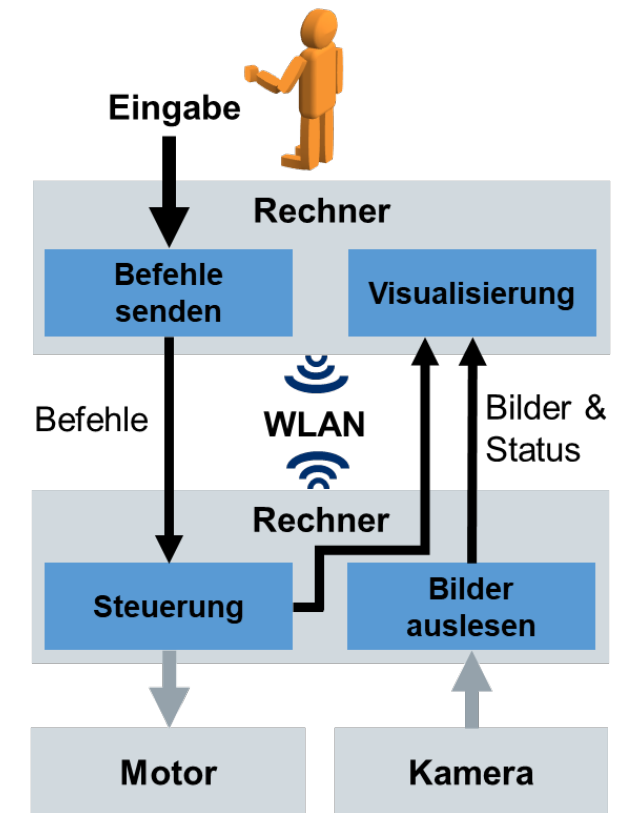
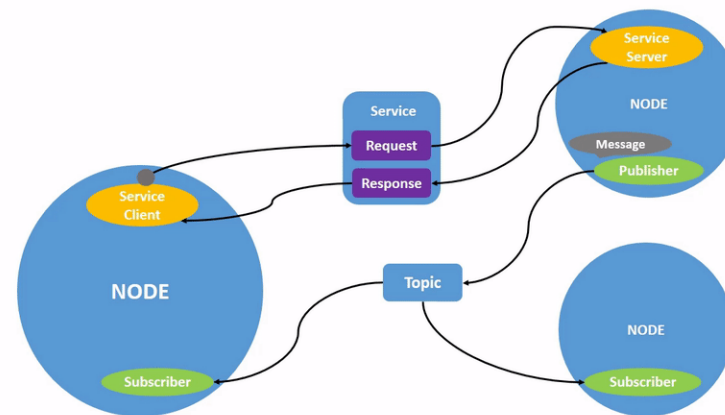
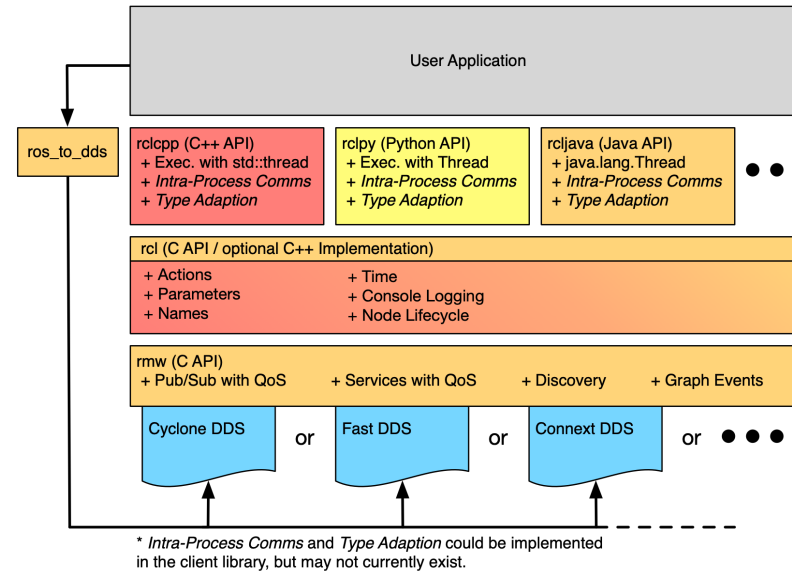


ROS



Nach dieser Vorlesungseinheit sollten folgende Fragen beantwortet werden können.

- Was ist das ROS?
- Warum wird Software in Packages aufgeteilt?
- Was ist ein ROS Node?
- Wie funktioniert die Kommunikation über Topics?
- Wie ist eine ROS Message aufgebaut?
- Wie funktioniert die Kommunikation mit ROS Services?
- Was sind Parameter im ROS?



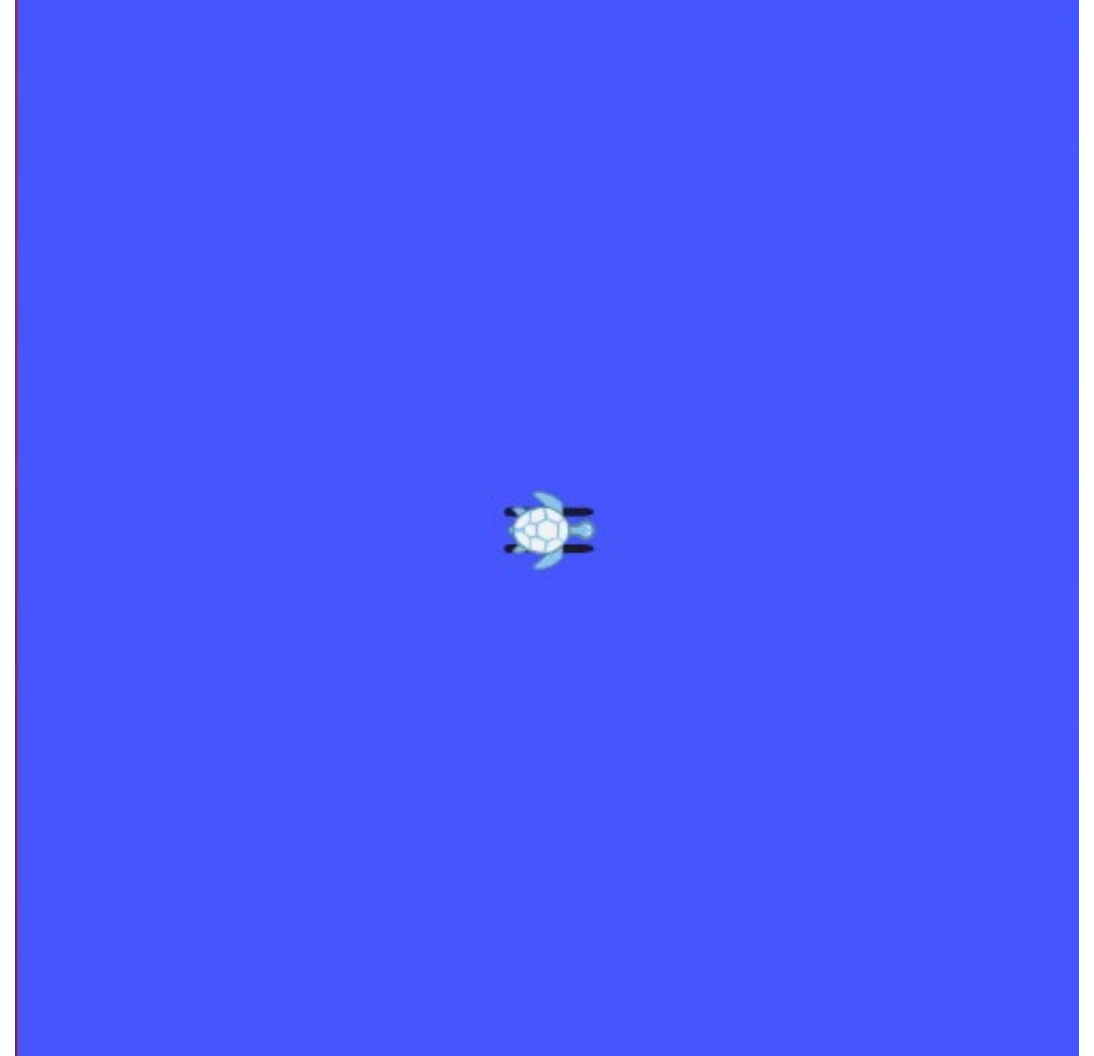
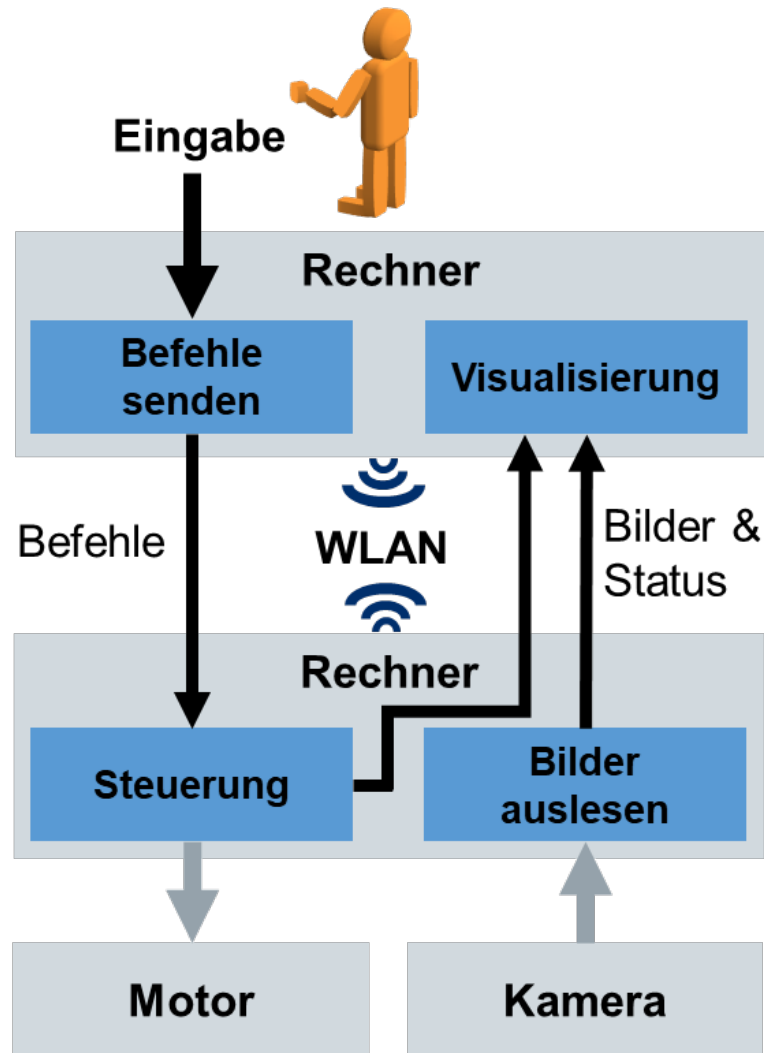
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- **ROS Actions**
- Packages der ROS-Community
- Koordinatensysteme und Transformationen: tf2
- Tools:
 - rqt
 - RViz
 - Ros2 launch
 - Ros2 bag



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Der mobile Roboter soll eine vorgegebene Trajektorie abfahren, also eine länger andauernde Handlung durchführen.



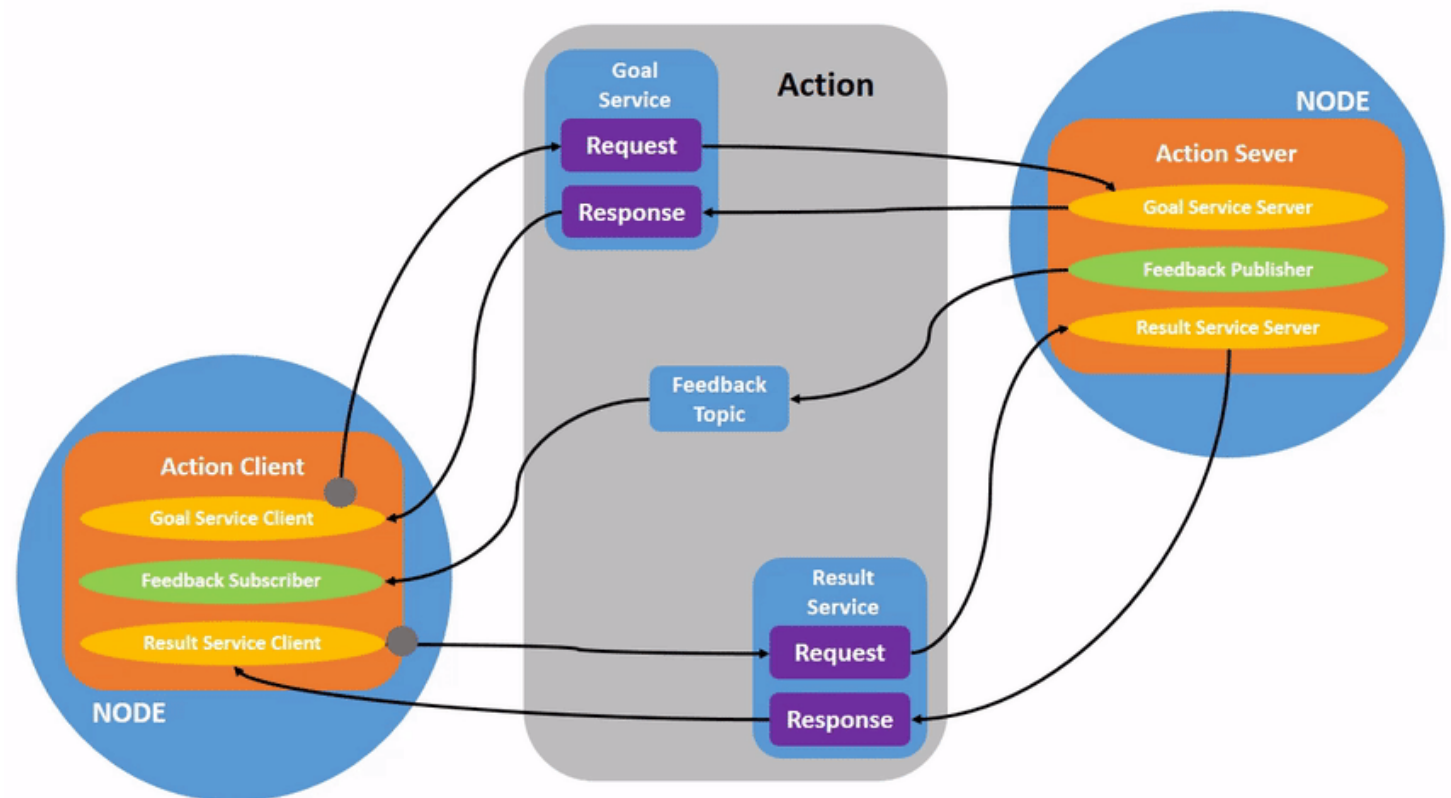
Die Kommunikation mit Actions erfolgt mithilfe von Services und Topics.

■ Action Server:

- Stellt die Action für andere Nodes bereit
- Akzeptiert oder lehnt die Goals eines oder mehrerer Action Clients ab
- Führt die Action aus, sobald ein Goal akzeptiert wurde
- Stellt Feedback (nutzerdefiniert) über die Aktionsausführung bereit (optional)
- Kann Goals / Aktionsausführungen abbrechen
- Sendet eine Rückmeldung über das Ergebnis (Result) der Action zum Client zurück (Erfolg, Abbruch, Fehlgeschlagen)

■ Action Client:

- Sendet Goals zum Action Server
- Kann das Feedback des Action Servers zu den Zielen überwachen
- Kann den Status der Goals des Action Servers überwachen
- Kann den Action Server zum Abbruch eines aktiven Ziels auffordern
- Kann das Ergebnis eines Ziels des Action Servers erhalten



<http://design.ros2.org/articles/actions.html>
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>

Actions werden mittels Interface Definition Language (IDL) in .action-Dateien durch Messages für Goal, Result und Feedback definiert.

Definition in .action-Datei (ROS IDL):

Goal

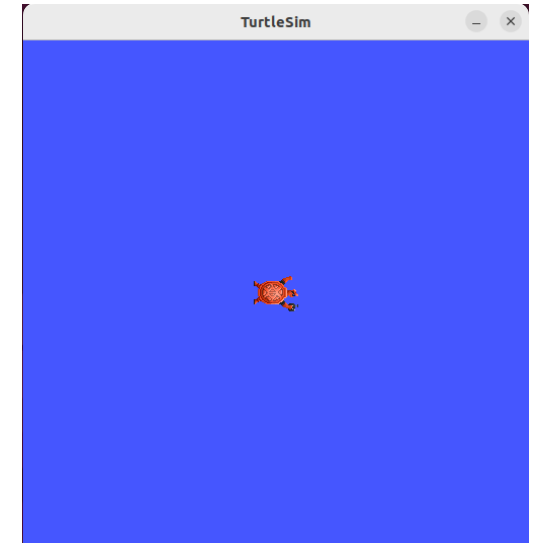
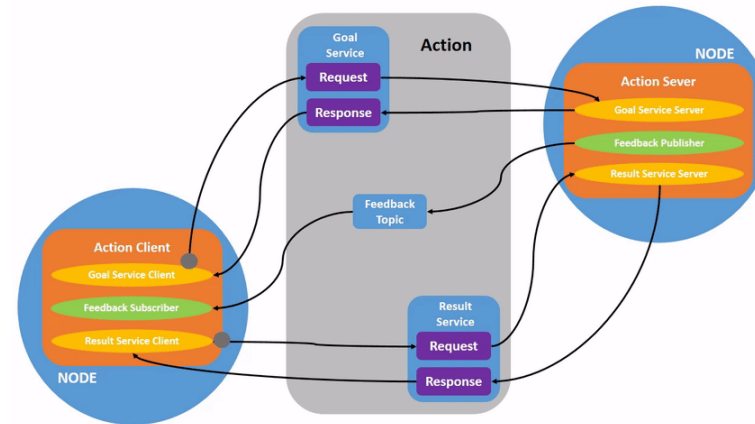
Beschreibt das Ziel einer Action. Wird vom Client zum Server für den Start der Action gesandt.

Result

Beschreibt das Ergebnis einer Action. Wird nach Abschluss (Erfolg, Abbruch, Fehlschlag) vom Server zum Client gesandt.

Feedback

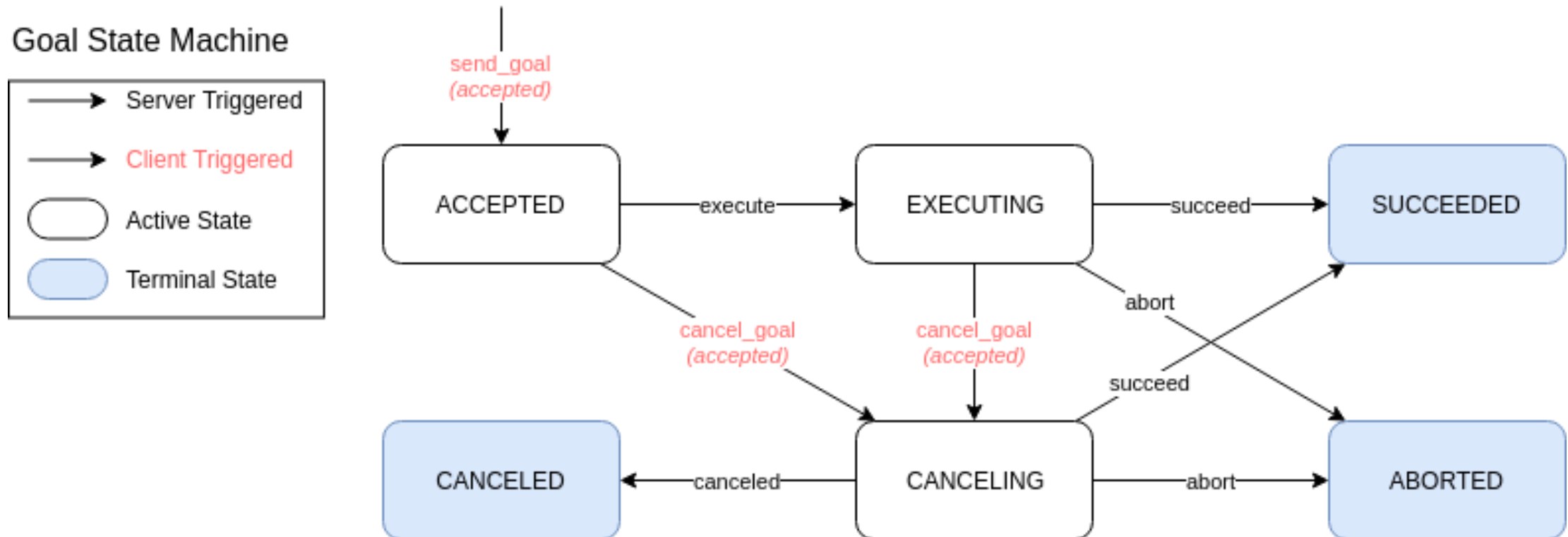
Beschreibt den Fortschritt einer Action. Wird vom Server zum Client während der Ausführung der Action gesandt.



```
gdr@gdr-VirtualBox:~$ ros2 interface show turtlesim/action/RotateAbsolute
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

<http://design.ros2.org/articles/actions.html>
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>

Ein Goal kann vom Action Server abgelehnt werden (rejected) oder entsprechend der Goal State Machine weiterbearbeitet werden.



<http://design.ros2.org/articles/actions.html>

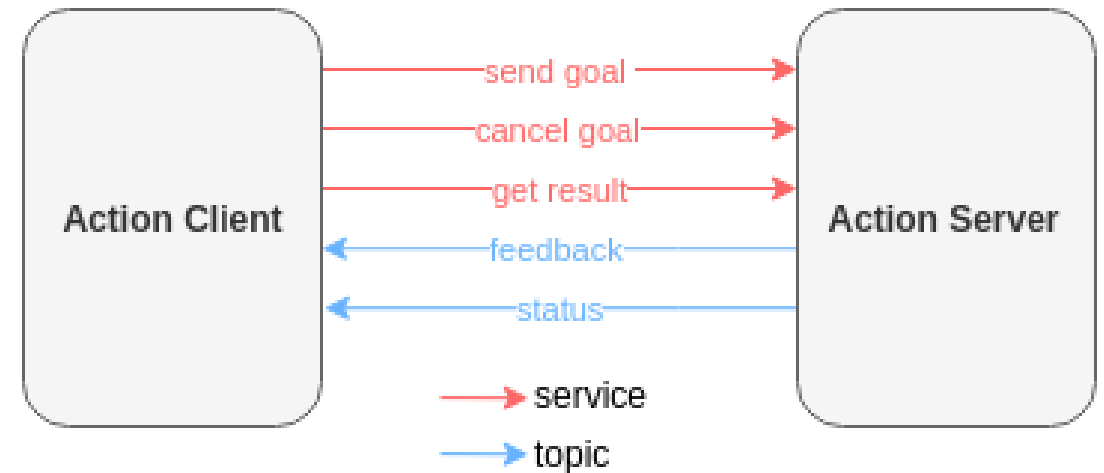
Actions können mittel eines Cancel-Services gezielt abgebrochen werden.

■ Send Goal Service:

- Client sendet Request an Server
- Request: Goal-Message, optional: universally unique identifier (UUID, Goal ID)
- Response: Antwort, ob das Goal akzeptiert oder abgelehnt wurde und Zeitstempel, falls das Goal akzeptiert wurde

■ Cancel Goal Service:

- Client sendet Request an Server
- Request: Goal ID und Zeitstempel
- Response: Response-Code und Liste mit Goals, die in den CANCELING-Zustand übergegangen sind
- Response-Code: Gibt Probleme beim Verarbeiten des Requests an (z.B. Ok, Abgelehnt, ungültige Goal ID)
- Übergang des CANCELED-Zustand über Status-Topic einsehbar



	Keine Goal ID	Goal ID angegeben
Kein Zeitstempel	Alle Goals werden abgebrochen	Goal mit angegebener ID wird abgebrochen
Zeitstempel angegeben	Alle Goals, die vor dem Zeitstempel akzeptiert wurden, werden abgebrochen	Goal mit angegebener ID und alle Goals, die vor dem Zeitstempel akzeptiert wurden, werden abgebrochen

<http://design.ros2.org/articles/actions.html>
https://en.wikipedia.org/wiki/Universally_unique_identifier

Der Fortschritt einer Action wird kontinuierlich über das Feedback-Topic ausgegeben.

■ Get Result Service:

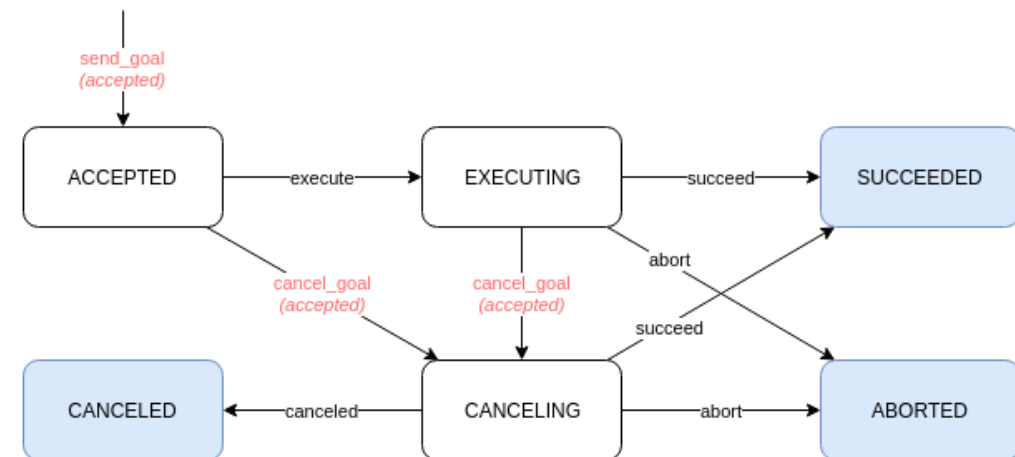
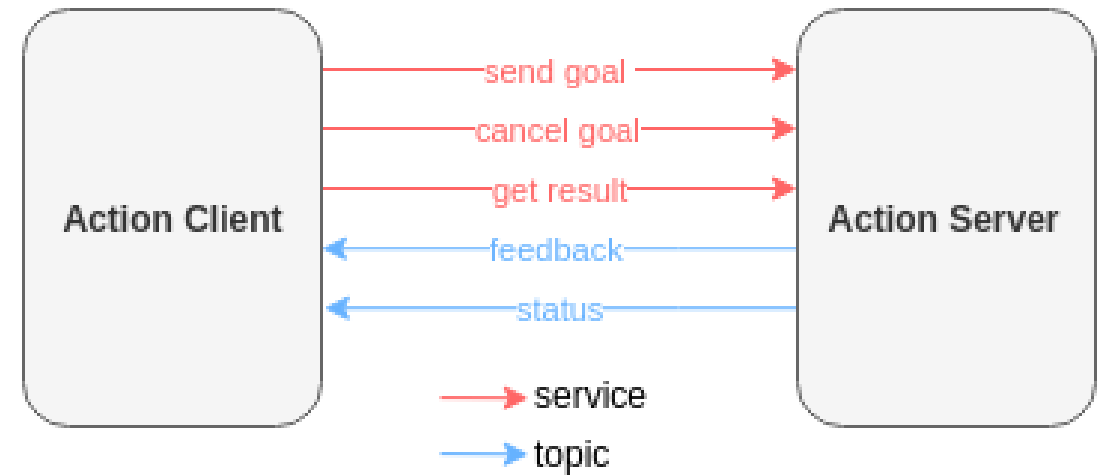
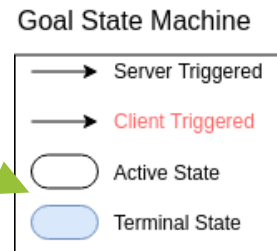
- Client sendet Request an Server
- Request: Goal ID
- Response: Status des Goals und Result-Message
- Results sollten vom Server zwischengespeichert werden

■ Goal Status Topic:

- Publisher des Servers
- Liste mit Zuständen der aktuell am Server vorhandenen Goals

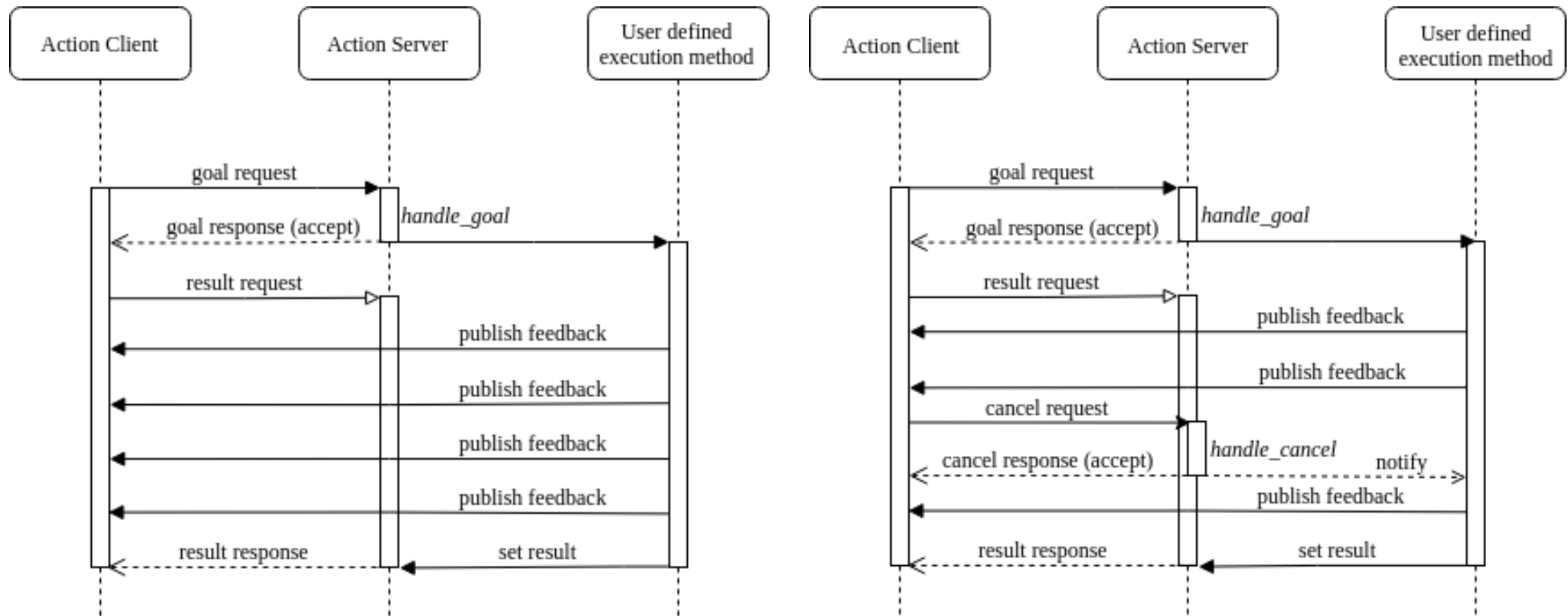
■ Feedback Topic:

- Publisher des Servers
- Goal ID, Feedback-Message
- Frequenz kann frei gewählt werden



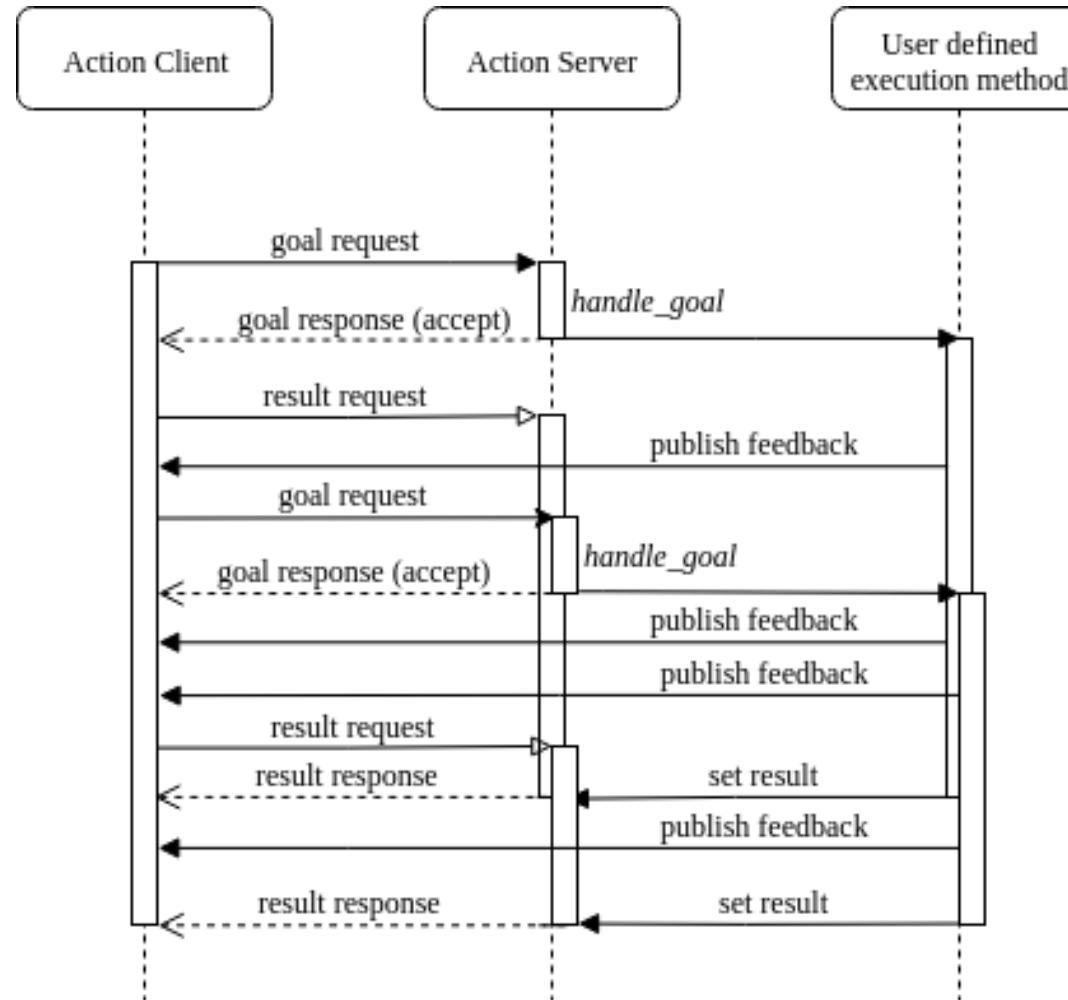
<http://design.ros2.org/articles/actions.html>

Actions sind für komplexe, langandauernde Aktionen und Prozesse gut geeignet.



<http://design.ros2.org/articles/actions.html>

Die Response-Message des Result-Services wird nach Beendigung einer Action versandt.



<http://design.ros2.org/articles/actions.html>

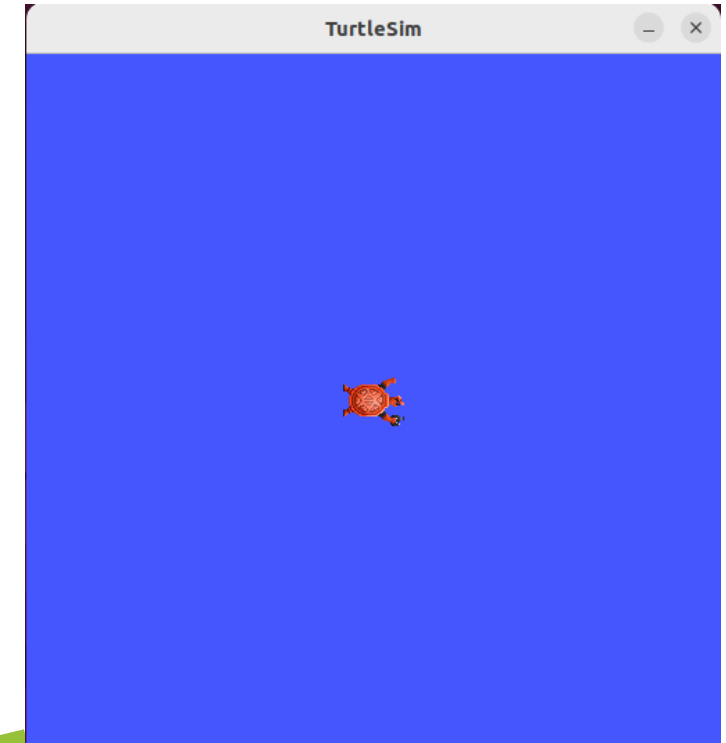
Der Turtlesim stellt einen einfachen Action Server zur Verfügung.

```
gdr@gdr-VirtualBox:~$ ros2 interface list
Messages:
  action_msgs/msg/GoalInfo
  action_msgs/msg/GoalStatus
  action_msgs/msg/GoalStatusArray
  actionlib_msgs/msg/GoalID
  actionlib_msgs/msg/GoalStatus
  actionlib_msgs/msg/GoalStatusArray
  builtin_interfaces/msg/Duration
  builtin_interfaces/msg/Time
```

...

```
Actions:
  action_tutorials_interfaces/action/Fibonacci
  example_interfaces/action/Fibonacci
  tf2_msgs/action/LookupTransform
  turtlesim/action/RotateAbsolute
```

```
gdr@gdr-VirtualBox:~$ ros2 action list
/turtle1/rotate_absolute
gdr@gdr-VirtualBox:~$ ros2 action info /turtle1/rotate_absolute
Action: /turtle1/rotate_absolute
Action clients: 0
Action servers: 1
  /turtlesim
```



```
gdr@gdr-VirtualBox:~$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

Die Topics und Services von Actions sind versteckt.

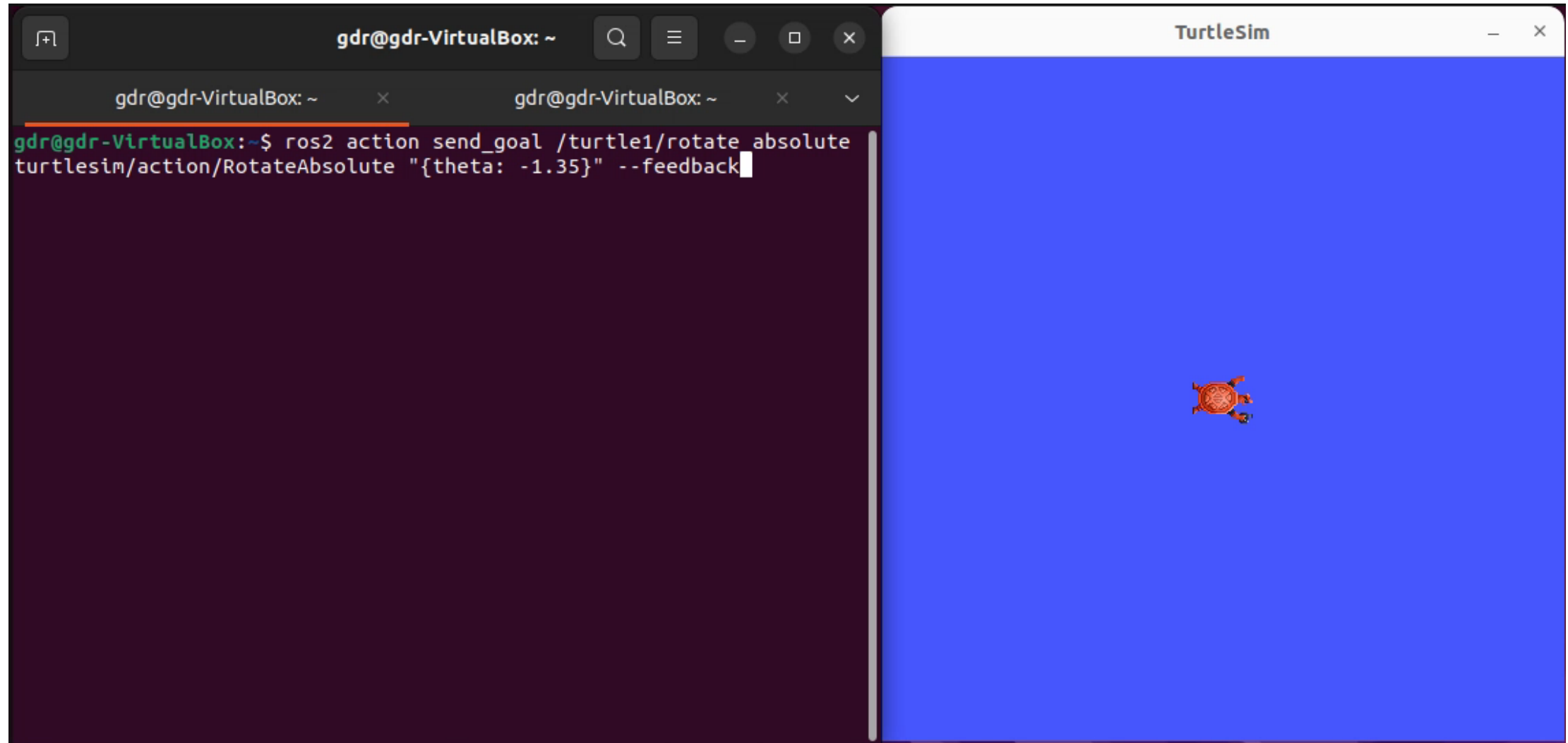
```
gdr@gdr-VirtualBox:~$ ros2 topic list --include-hidden-topics
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/turtle1/rotate_absolute/_action/feedback
/turtle1/rotate_absolute/_action/status
gdr@gdr-VirtualBox:~$ ros2 service list --include-hidden-services
/clear
/kill
/reset
/spawn
/turtle1/rotate_absolute/_action/cancel_goal
/turtle1/rotate_absolute/_action/get_result
/turtle1/rotate_absolute/_action/send_goal
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
```

```
gdr@gdr-VirtualBox:~$ ros2 interface show action_msgs/srv/CancelGoal
# Cancel one or more goals with the following policy:
#
# - If the goal ID is zero and timestamp is zero, cancel all goals.
# - If the goal ID is zero and timestamp is not zero, cancel all goals accepted
#   at or before the timestamp.
# - If the goal ID is not zero and timestamp is zero, cancel the goal with the
#   given ID regardless of the time it was accepted.
# - If the goal ID is not zero and timestamp is not zero, cancel the goal with
#   the given ID and all goals accepted at or before the timestamp.
#
# Goal info describing the goals to cancel, see above.
GoalInfo goal_info
  unique_identifier_msgs/UUID goal_id
  #
  uint8[16] uuid
  builtin_interfaces/Time stamp
  int32 sec
  uint32 nanosec
---
```

```
gdr@gdr-VirtualBox:~$ ros2 interface package action_msgs
action_msgs/msg/GoalStatusArray
action_msgs/msg/GoalStatus
action_msgs/srv/CancelGoal
action_msgs/msg/GoalInfo
```

```
gdr@gdr-VirtualBox:~$ ros2 service --include-hidden-services type /turtle1/rotate_absolute/_action/cancel_goal
action_msgs/srv/CancelGoal
```

Mithilfe von `ros2 action` können neue Goals an einen Action Server gesandt werden.



Mithilfe von `ros2 action` können neue Goals an einen Action Server gesandt werden.

```
gdr@gdr-VirtualBox:~$ ros2 action send_goal /turtle1/rotate_absolute
turtlesim/action/RotateAbsolute "{theta: -1.35}"
Waiting for an action server to become available...
Sending goal:
  theta: -1.35

Goal accepted with ID: b90901e9013846d6adf86a1c3e2a46c4

Result:
  delta: 1.3600000143051147

Goal finished with status: SUCCEEDED
```

Das ROS bietet Kommunikationsmöglichkeiten für unterschiedliche Einsatzzwecke.

	(Parameter)	Topics	Services	Actions
Beschreibung	Speicherung von Parametern für Nodes, andere Nodes können mit den Parameter interagieren	Unidirektionale periodische Peer-to-Peer Kommunikation nach dem Publisher/Subscriber-Prinzip	Bidirektionale Peer-to-Peer Kommunikation; Aufruf eines Services erfolgt durch Request-Message, Ergebnis der Bearbeitung wird durch Response-Message zurückgesendet; Kein (externer) Abbruch möglich	Bidirektionale Kommunikation auf Basis von Topics und Services; Action startet durch Senden eines Goals; Während der Ausführung wird Feedback versendet; Bei Beendigung Versenden eines Results; Abbruch möglich
Anwendung	Grundlegende Einstellungen / Werte eines Nodes	Sensordaten, Befehle, Status	Kurze, aperiodische Kommunikation	Langandauernde, komplexe Handlungen
Beispiel	Maximale Geschwindigkeit: max_vel = 0.5	Versenden der aktuellen Pose des Roboters	Hardware an-/abschalten	Navigation
Struktur / Definition	▪ Key: String ▪ Value: Datenstruktur (z.B. String, Double) ▪ Descriptor (optional): Beschreibung des Parameters	.msg-Datei: Header (std_msgs, optional), Datenstruktur (built-in Type, andere Message Types)	.srv-Datei: Request (Message) --- Response (Message)	.action-Datei: Goal (Message) --- Result (Message) --- Feedback (Message)

<https://docs.ros.org/en/humble/How-To-Guides/Topics-Services-Actions.html>

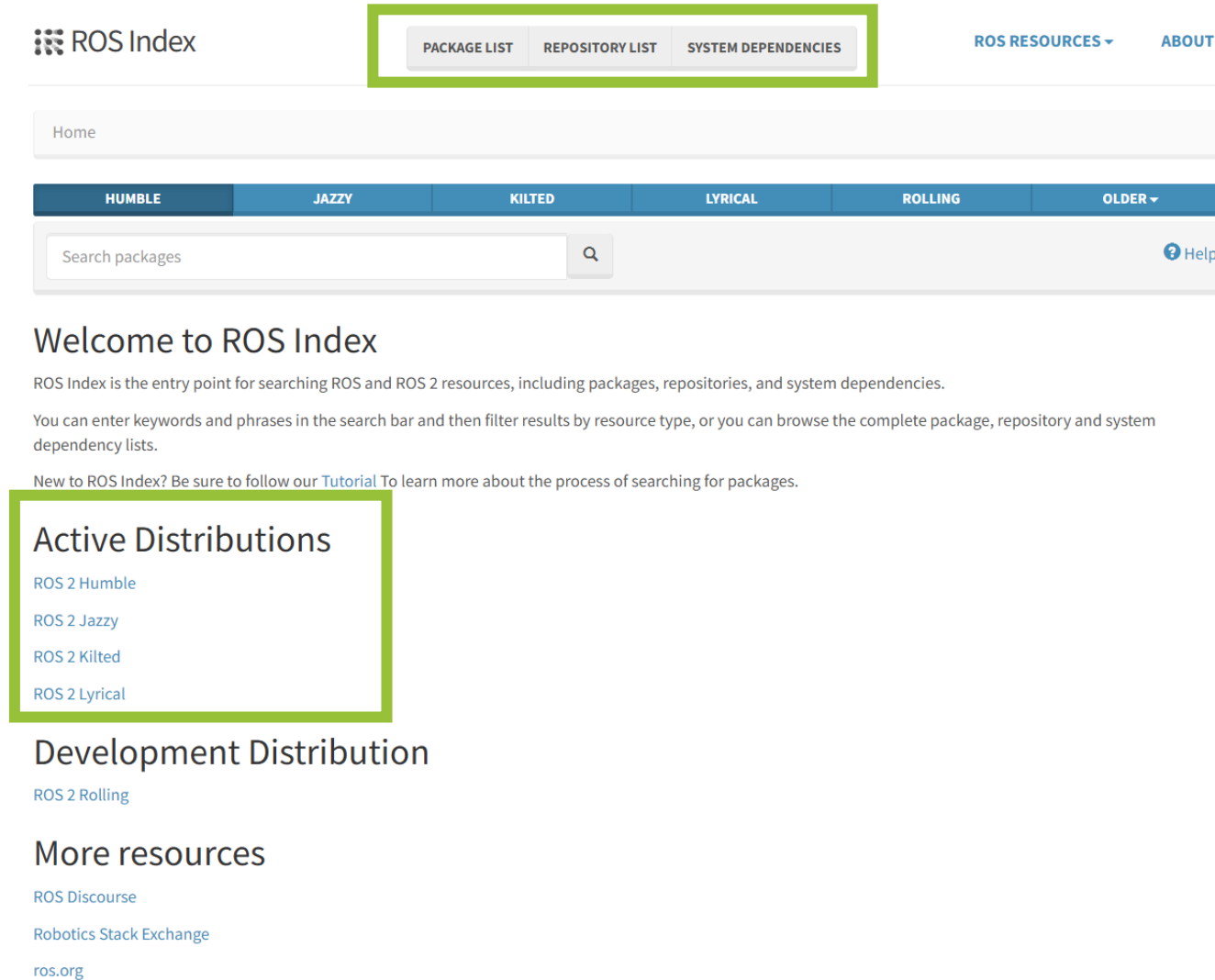
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- ROS Actions
- **Packages der ROS-Community**
- Koordinatensysteme und Transformationen: tf2
- Tools:
 - rqt
 - RViz
 - Ros2 launch
 - Ros2 bag



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Im ROS Index können verfügbare ROS Packages und Repositories für die unterschiedlichen Distributionen gefunden werden.



ROS Index

PACKAGE LIST REPOSITORY LIST SYSTEM DEPENDENCIES

ROS RESOURCES ▾ ABOUT ▾

Home

HUMBLE JAZZY KILTED LYRICAL ROLLING OLDER ▾

Search packages 🔍 [? Help](#)

Welcome to ROS Index

ROS Index is the entry point for searching ROS and ROS 2 resources, including packages, repositories, and system dependencies.

You can enter keywords and phrases in the search bar and then filter results by resource type, or you can browse the complete package, repository and system dependency lists.

New to ROS Index? Be sure to follow our [Tutorial](#) To learn more about the process of searching for packages.

Active Distributions

- [ROS 2 Humble](#)
- [ROS 2 Jazzy](#)
- [ROS 2 Kilted](#)
- [ROS 2 Lyrical](#)

Development Distribution

- [ROS 2 Rolling](#)

More resources

- [ROS Discourse](#)
- [Robotics Stack Exchange](#)
- [ros.org](#)

<https://index.ros.org/>

Mithilfe der Suche können spezifische Packages gefunden werden.

ROS Resources: [Documentation](#) | [Support](#) | [Discussion Forum](#) | [Service Status](#) | [Q&A answers.ros.org](#)

ROS Index [ABOUT](#) [INDEX](#) [CONTRIBUTE](#) [STATS](#)

Home > Search

Suche nach Packages mit dem Wort „cam“

PACKAGES SYSTEM DEPENDENCIES

Package search results

⚡	📄	🕒	Distro	Name	Repo
		2022-09-27	foxy	ueye_cam	github-anqixu-ueye_cam
⚡		2023-04-03	foxy	usb_cam	github-ros-drivers-usb_cam
⚡		2023-04-03	humble	usb_cam	github-ros-drivers-usb_cam
⚡		2023-04-03	iron	usb_cam	github-ros-drivers-usb_cam
⚡		2023-04-03	rolling	usb_cam	github-ros-drivers-usb_cam
		2022-03-08	noetic	ueye_cam	github-anqixu-ueye_cam
⚡		2023-05-26	noetic	usb_cam	github-ros-drivers-usb_cam
		2023-04-04	noetic	speed_cam	github-nasa-astrobee
⚡		2020-07-12	noetic	usb_cam_controllers	github-yoshito-n-students-usb_cam_hardware
⚡		2020-07-12	noetic	usb_cam_hardware	github-yoshito-n-students-usb_cam_hardware

Ausschnitt des Suchergebnisses

« 1 2 3 4 »

<https://index.ros.org/>

Für jedes erfasste Package existiert eine Übersichtsseite mit Informationen zum Package.

The screenshot shows the ROS Package Overview page for the `usb_cam` package. At the top, a navigation bar allows selecting the ROS distribution, with `humble` currently selected. The page header identifies the package as `usb_cam` from the `usb_cam` repo, associated with the `humble` ROS Distro. On the right, links for `API Docs` and `Browse Code` are provided. Below the header, a tabbed interface shows `Overview`, `1 Assets`, `21 Dependencies`, and `>50 Q & A`. The `Overview` tab is active, displaying a `Package Summary` on the left and a `Package Description` on the right. The `Package Summary` includes details such as Version (0.8.1), License (BSD), Build type (AMENT_CMAKE), and Use (RECOMMENDED). It also features a `Repository Summary` with fields for Checkout URI, VCS Type (git), VCS Version (main), Last Updated (2026-02-26), Dev Status (MAINTAINED), Released (RELEASED), and Contributing information (Help Wanted (1), Good First Issues (0), Pull Requests to Review (5)). The `Package Description` states it is a ROS Driver for V4L USB Cameras and lists additional links (Website, Bugtracker, Repository), maintainers (Evan Flynn), and authors (No additional authors).

Auswahl der Distribution

Informationen zum Package

https://index.ros.org/p/usb_cam/github-ros-drivers-usb_cam/#humble

Die README eines Packages sollte die Installation, Funktionsweise und Nutzung des Packages beschreiben.

README.md

usb_cam ROS2 CI passing

A ROS2 Driver for V4L USB Cameras

This package is based off of V4L devices specifically instead of just UVC.

For ros1 documentation, see [the ROS wiki](#).

Supported ROS2 Distro and Platforms

All Officially supported Linux Distro and corresponding ROS2 releases are supported. Please create an issue if you experience any problems on these platforms.

Windows: TBD/Untested/Unproven MacOS: TBD/Untested/Unproven

For either MacOS or Windows - if you would like to try and get it working please create an issue to document your effort. If it works we can add it to the instructions here!

Quickstart

Assuming you have a supported ROS2 distro installed, run the following command to install the binary release:

```
sudo apt get install ros-<ros2-distro>-usb_cam
```

As of today this package should be available for binary installation on all active ROS2 distros.

If for some reason you cannot install the binaries, follow the directions below to compile from source.

```
gdr@gdr-VirtualBox:~$ sudo apt install ros-humble-usb_cam
[sudo] password for gdr:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ffmpeg libass9 libavdevice58 libavfilter7 libbs2b0 libfl
  libmysofa1 libopenal-data libopenal1 libpocketsphinx3 li
  librubberband2 libserd-0-0 libsord-0-0 libsphinxbase3 li
  libv4l2rds0 libvidstab1.1 libzing2 pocketsphinx-en-us
  ros-humble-camera-calibration-parsers ros-humble-camera-
  ros-humble-compressed-depth-image-transport
  ros-humble-compressed-image-transport ros-humble-image-t
  ros-humble-theora-image-transport v4l-utils
Suggested packages:
  ffmpeg-doc libportaudio2 serdi sordi
The following NEW packages will be installed:
  ffmpeg libass9 libavdevice58 libavfilter7 libbs2b0 libfl
  libmysofa1 libopenal-data libopenal1 libpocketsphinx3 li
  librubberband2 libserd-0-0 libsord-0-0 libsphinxbase3 li
  libv4l2rds0 libvidstab1.1 libzing2 pocketsphinx-en-us
  ros-humble-camera-calibration-parsers ros-humble-camera-
  ros-humble-compressed-depth-image-transport
  ros-humble-compressed-image-transport ros-humble-image-t
```

Installation mit Ubuntu-Paketmanager

https://index.ros.org/p/usb_cam/github-ros-drivers-usb_cam/#humble

Die README eines Packages sollte die Installation, Funktionsweise und Nutzung des Packages beschreiben.

Building from Source

Clone/Download the source code into your workspace:

```
cd /path/to/catkin_ws/src
git clone https://github.com/ros-drivers/usb_cam.git
```

Or click on the green "Download zip" button on the repo's github webpage.

Once downloaded and ensuring you have sourced your ROS2 underlay, go ahead and install the dependencies:

```
cd /path/to/colcon_ws
rosdep install --from-paths src --ignore-src -y
```

From there you should have all the necessary dependencies installed to compile the `usb_cam` package:

```
cd /path/to/colcon_ws
colcon build
source /path/to/colcon_ws/install/setup.bash
```

Be sure to source the newly built packages after a successful build.

Once sourced, you should be able to run the package in one of three ways, shown in the next section.

→ Installation im eigenen Workspace

https://index.ros.org/p/usb_cam/github-ros-drivers-usb_cam/#humble

Der Quellcode von ROS Packages ist meist auf GitHub zu finden.

ros-drivers / **usb_cam** Public

Notifications Fork 538

<> Code Issues 7 Pull requests 3 Discussions Actions Projects Security Insights

ros2 5 branches 27 tags

This branch is 130 commits ahead, 56 commits behind develop.

flynn Remove debug print accidentally added ...

.github/workflows	Enable code coverage using loc...
config	Implement new pixel_format cl...
include/usb_cam	Clean up ROS 2 node, update p...
launch	Add basic linters to CMake, fix...
msg	adding launch file
scripts	Add basic linters to CMake, fix linter errors found 7 months ago
src	Remove debug print accidentally added 2 months ago
test	Fix linter errors, clean up tests 3 months ago
.gitignore	add service, install launch, separate header 2 years ago

Local Codespaces

Clone ?

HTTPS GitHub CLI

https://github.com/ros-drivers/usb_cam.git

Use Git or checkout with SVN using the web URL.

Open with GitHub Desktop

Download ZIP

About

A ROS Driver for V4L USB Cameras

wiki.ros.org/usb_cam

Readme

View license

396 stars

30 watching

538 forks

Report repository

Releases 3

Update pixel format logic, supp... on Apr 2

+ 2 releases

https://github.com/ros-drivers/usb_cam/tree/ros2

Der Node kann wie gewohnt gestartet werden.

Running

The `usb_cam_node` can be ran with default settings, by setting specific parameters either via the command line or by loading in a parameters file.

We provide a "default" params file in the `usb_cam/config/params.yaml` directory to get you started. Feel free to modify this file as you wish.

Also provided is a launch file that should launch the `usb_cam_node_exe` executable along with an additional node that displays an image topic.

The commands to run each of these different ways of starting the node are shown below:

NOTE: you only need to run ONE of the commands below to run the node

```
# run the executable with default settings (without params file)
ros2 run usb_cam usb_cam_node_exe
```

```
# run the executable while passing in parameters via a yaml file
ros2 run usb_cam usb_cam_node_exe --ros-args --params-file /path/to/colcon_ws/src/usb_cam/config/param
```

```
# launch the usb_cam executable that loads parameters from the same `usb_cam/config/params.yaml` file
# along with an additional image viewer node
ros2 launch usb_cam demo_launch.py
```

https://github.com/ros-drivers/usb_cam/tree/ros2

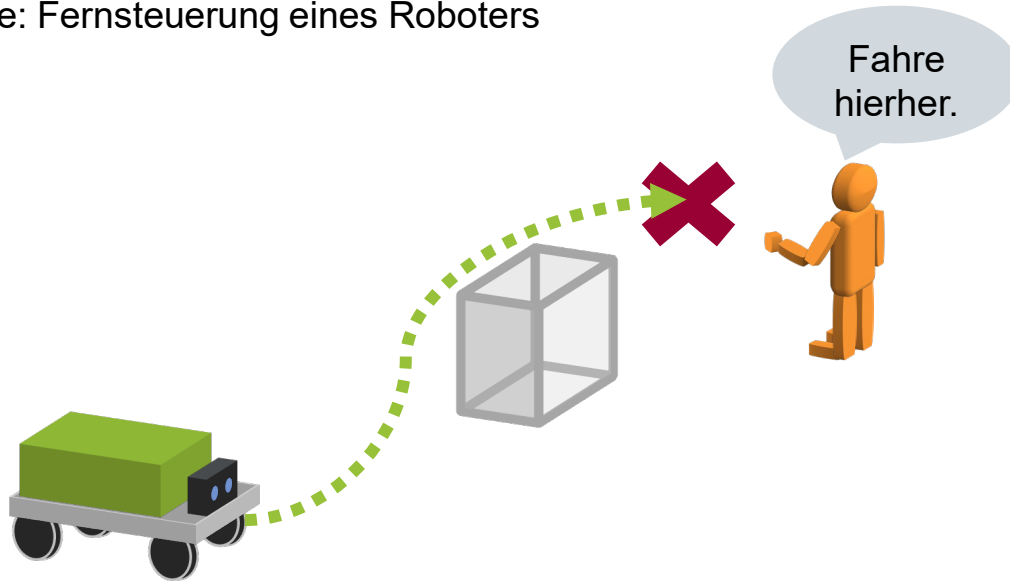
Für eine Vielzahl an Robotern, Sensoren, Aktoren und Algorithmen sind ROS-Packages bereits frei verfügbar.

```
gdr@gdr-VirtualBox:~$ ros2 run usb_cam usb_cam_node_exe
[INFO] [1687182754.288823499] [usb_cam]: camera_name value: default_cam
[WARN] [1687182754.288950251] [usb_cam]: framerate: 30.000000
Files [1687182754.291412373] [usb_cam]: using default calibration URL
[INFO] [1687182754.291516739] [usb_cam]: camera calibration URL: file:///home/gdr/.ros/camera_info/default_cam.yaml
[ERROR] [1687182754.291602935] [camera_calibration_parsers]: Unable to open camera calibration file [/home/gdr/.ros/camera_info/default_cam.yaml]
[WARN] [1687182754.291640075] [usb_cam]: Camera calibration file /home/gdr/.ros/camera_info/default_cam.yaml not found
[INFO] [1687182754.291681194] [usb_cam]: Starting 'default_cam' (/dev/video0) at 640x480 via mmap (yuyv) at 30 FPS
terminate called after throwing an instance of 'char*'
[ros2run]: Aborted
```

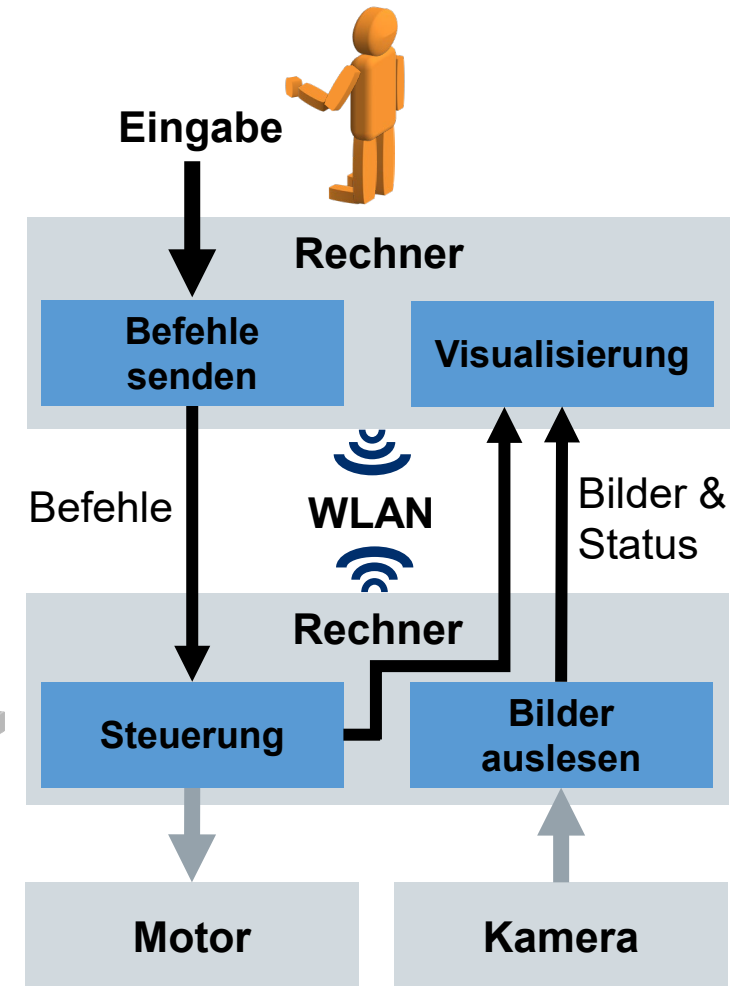
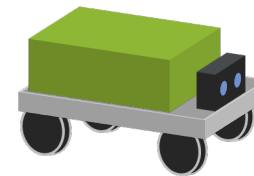


In dieser Vorlesungseinheit werden die grundlegenden Funktionen des ROS anhand eines einfachen Beispiels vorgestellt.

- Aufgabe: Fernsteuerung eines Roboters



ROS



Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

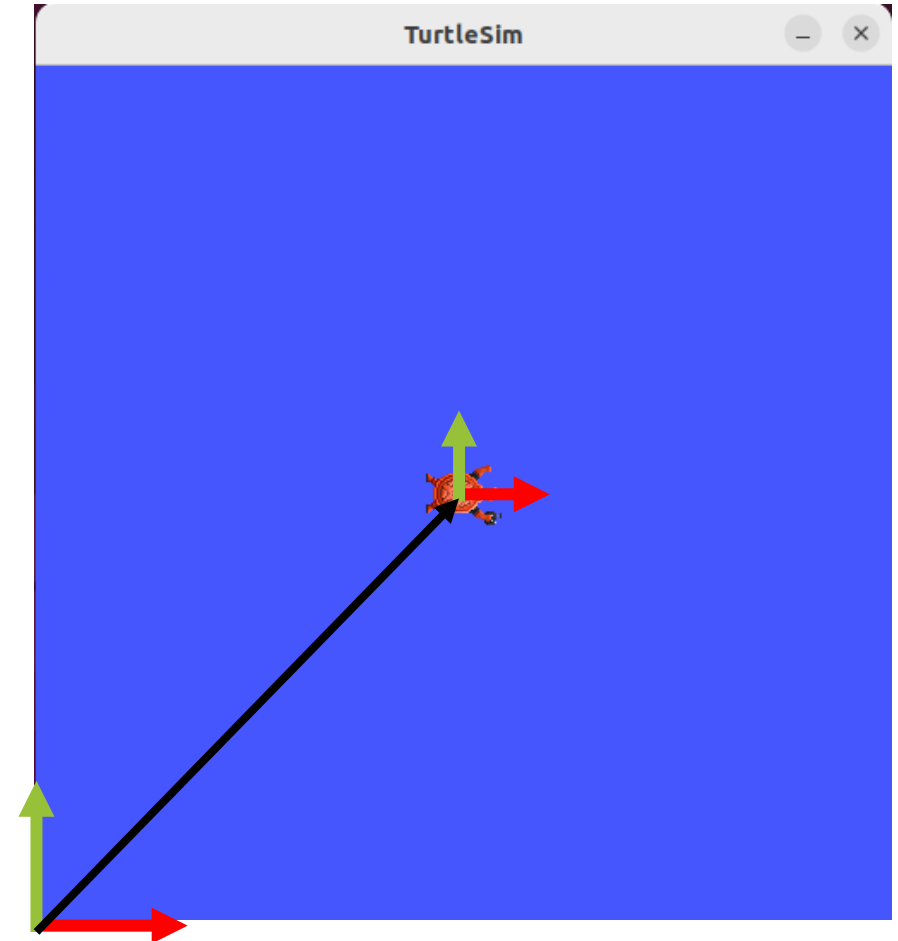
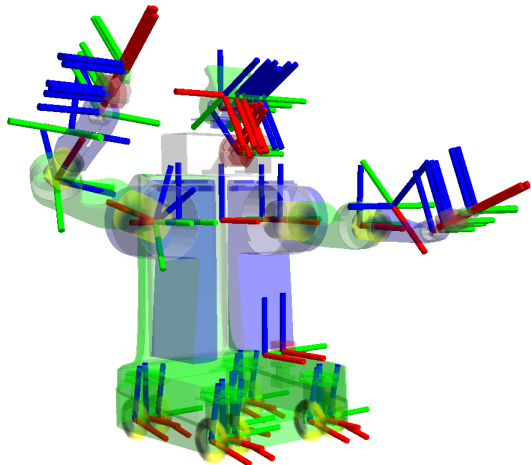
- ROS Actions
- Packages der ROS-Community
- **Koordinatensysteme und Transformationen: tf2**
- Tools:
 - rqt
 - RViz
 - Ros2 launch
 - Ros2 bag



<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

Mit dem ROS Package tf2 werden Transformationen zwischen unterschiedlichen Koordinatensystemen beschrieben.

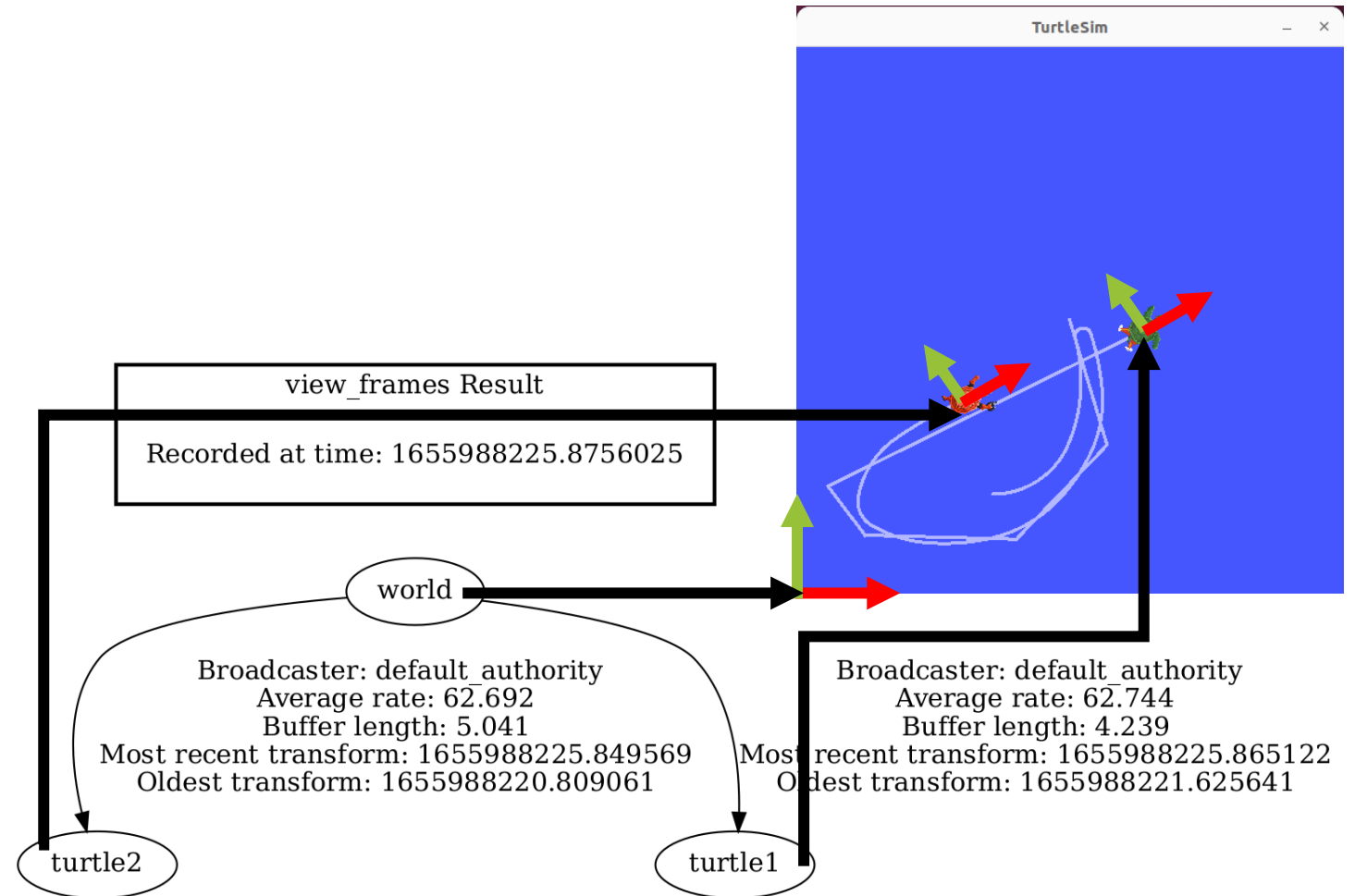
- Statische Transformation: keine Veränderung der Transformation mit der Zeit
 - Beispiel: Sensorkoordinatensystem zu Roboterbasiskoordinatensystem
- Dynamische Transformation: Änderung der Transformation mit der Zeit
 - Beispiel: Roboterbasiskoordinatensystem eines mobilen Roboters zu Weltkoordinatensystem



<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html>
<https://docs.ros.org/en/humble/Concepts/About-Tf2.html>

Koordinatensysteme und deren Transformationen können als Baum dargestellt werden.

- Ein Koordinatensystem in ROS kann maximal ein übergeordnetes Bezugskoordinatensystem (parent frame) haben
- Mehrere Koordinatensysteme (child frame) können dasselbe Bezugskoordinatensystem besitzen
- Es sind keine geschlossenen Schleifen im Baum möglich
- Die Transformationen werden auf den Topics /tf und /tf_static versendet



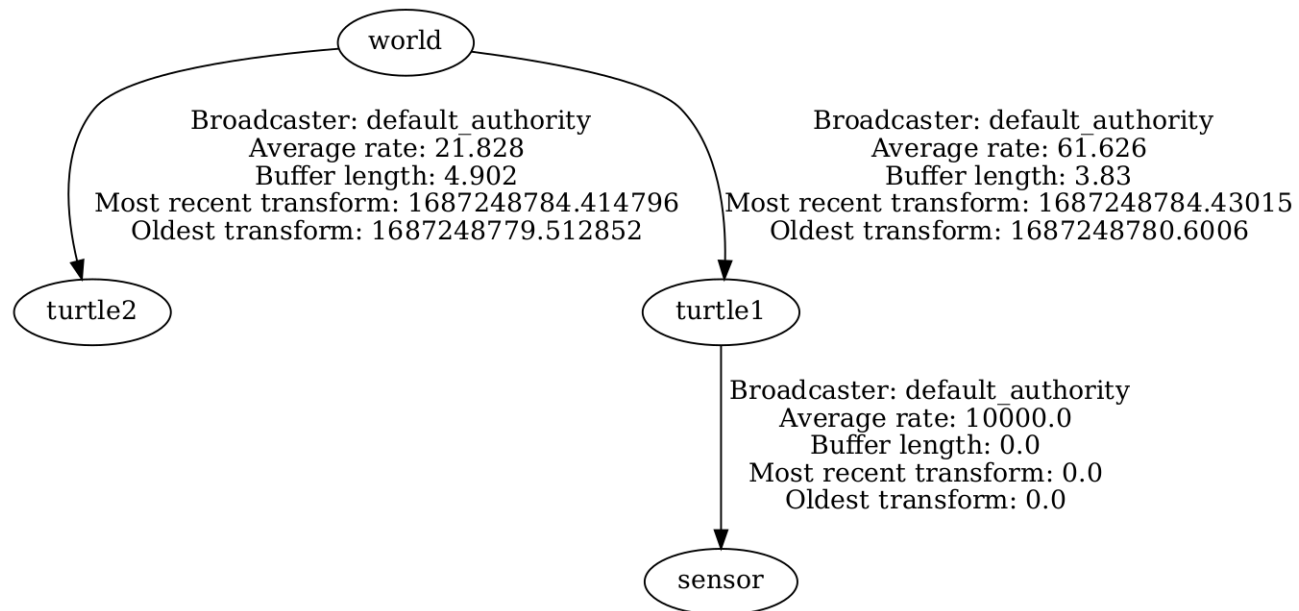
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html>
<https://docs.ros.org/en/humble/Concepts/About-Tf2.html>
<https://ros.org/reps/rep-0105.html>

Statische Transformationen können im Terminal definiert und versendet werden.

```
gdr@gdr-VirtualBox:~$ ros2 run tf2_ros static_transform_publisher --x 0.0 --y 0.0 --z 0.1
--roll 0.0 --pitch 0.0 --yaw 0.0 --frame-id turtle1 --child-frame-id sensor
[INFO] [1687248722.657012782] [static_transform_publisher_kcNjJotAtvbQfJBT]: Spinning until
stopped - publishing transform
translation: ('0.000000', '0.000000', '0.100000')
rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
from 'turtle1' to 'sensor'
```

view_frames Result

Recorded at time: 1687248784.437748



```
gdr@gdr-VirtualBox:~$ ros2 topic list
/parameter_events
/rosout
/tf
/tf_static
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/turtle2/cmd_vel
/turtle2/color_sensor
/turtle2/pose
```

```
gdr@gdr-VirtualBox:~$ ros2 topic info /tf
Type: tf2_msgs/msg/TFMessage
Publisher count: 2
Subscription count: 1
gdr@gdr-VirtualBox:~$ ros2 topic info /tf_static
Type: tf2_msgs/msg/TFMessage
Publisher count: 1
Subscription count: 1
```

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html>

Transformationen basieren auf dem Message Type TransformStamped.

```
gdr@gdr-VirtualBox:~$ ros2 interface show tf2_msgs/msg/TFMessage
geometry_msgs/TransformStamped[] transforms
#
#
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
string child_frame_id
Transform transform
  Vector3 translation
    float64 x
    float64 y
    float64 z
  Quaternion rotation
    float64 x 0
    float64 y 0
    float64 z 0
    float64 w 1
```

Compact Message Definition

```
std_msgs/msg/Header header
string child_frame_id
geometry_msgs/msg/Transform transform
```

Compact Message Definition

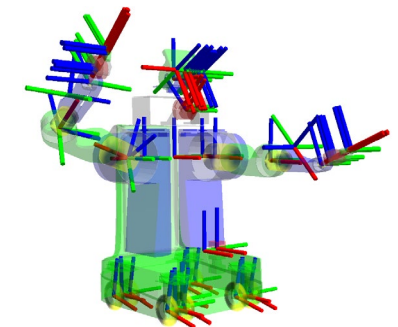
```
geometry_msgs/msg/Vector3 translation
geometry_msgs/msg/Quaternion rotation
```

Statische Transformationen:

- Basis eines Industrieroboters zu Weltkoordinatensystem
- Statische Hindernisse in einer Karte
- Festangebrachte Sensoren auf einem mobilen Roboter zum Basiskoordinatensystem des Roboters

Dynamische Transformationen:

- Basiskoordinatensystem eines mobilen Roboters zu Welt-/Kartenkoordinatensystem
- Achsenkoordinatensystem eines Industrieroboters zur Roboterbasis
- Mobiler Roboter zu anderem mobilem Roboter



https://docs.ros2.org/latest/api/geometry_msgs/msg/TransformStamped.html
https://docs.ros2.org/latest/api/geometry_msgs/msg/Transform.html
https://github.com/ros2/common_interfaces/tree/master/geometry_msgs

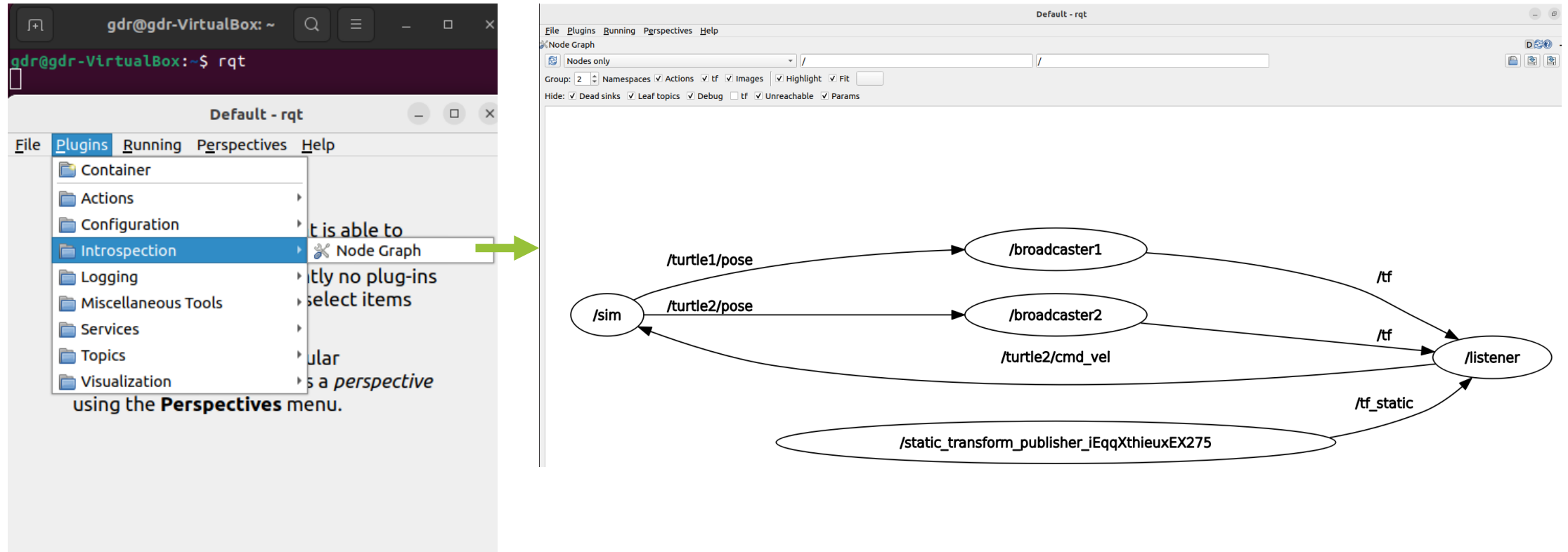
Das Robot Operating System ist ein Meta-Betriebssystem zur Entwicklung von Software für Roboter und zum Betrieb von Robotern.

- ROS Actions
- Packages der ROS-Community
- Koordinatensysteme und Transformationen: tf2
- **Tools:**
 - rqt
 - RViz
 - Ros2 launch
 - Ros2 bag

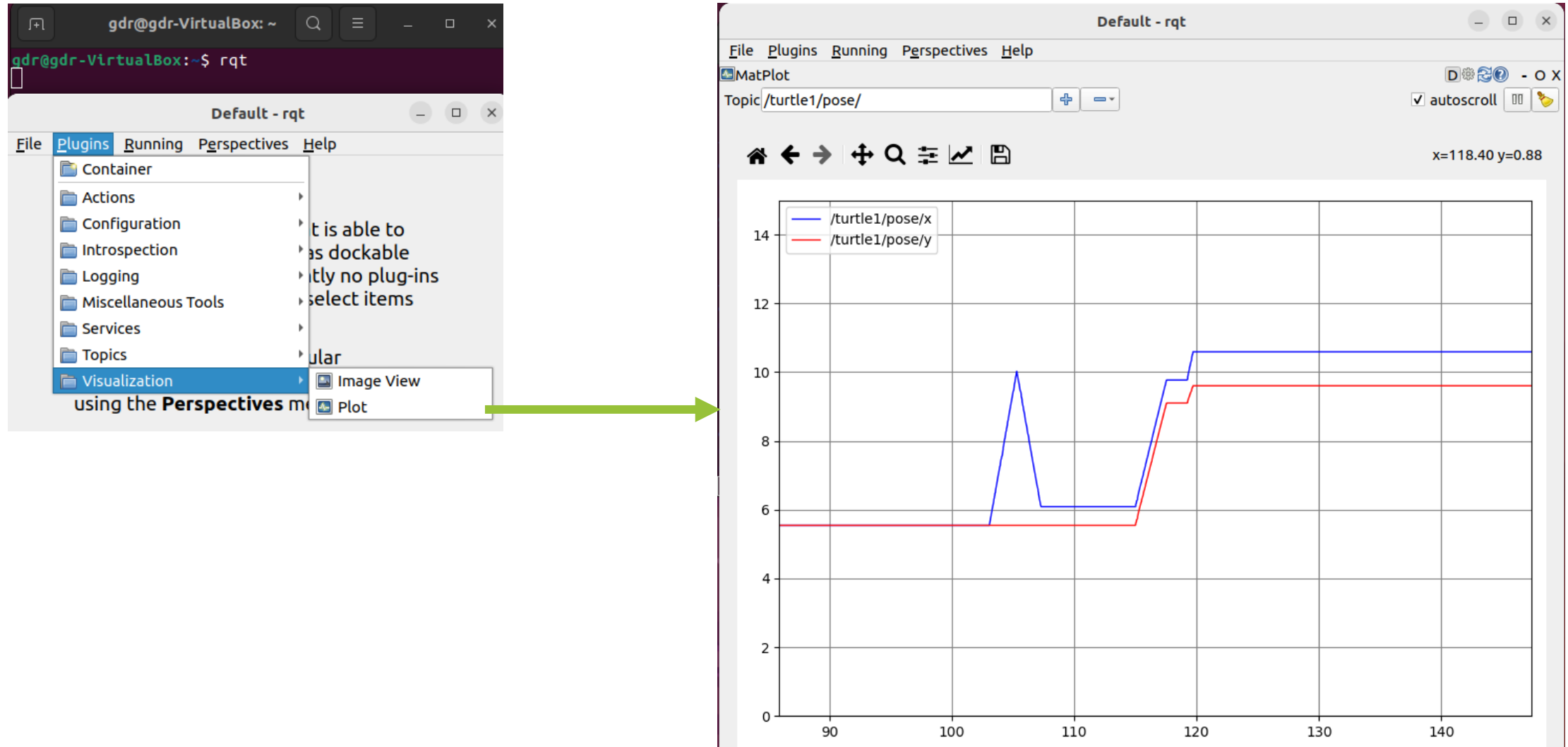


<https://github.com/ros-infrastructure/artwork/blob/master/distributions/humble/HumbleHawksbill.png>

rqt ist eine Sammlung von Werkzeugen zur Untersuchung, Visualisierung und Interaktion.



Mit rqt können die Daten von Topics geplottet werden.



Mit dem Tool Service Caller können Request-Messages an Services gesendet werden.

The image shows the ROS Service Caller tool interface. On the left, the 'Plugins' menu is open, and 'Service Caller' is selected. The main window displays the 'Service Caller' tab with the service '/spawn' selected. The 'Request' section shows a table with the following data:

Topic	Type	Expression
/spawn	turtlesim/srv/Spawn	
x	float	1.0
y	float	2.0
theta	float	1.57
name	string	'gdr_turtle'

The 'Response' section shows a table with the following data:

Field	Type	Value
/	turtlesim/srv/Spawn.Response	
name	string	'gdr_turtle'

The right side of the image shows the 'TurtleSim' window with a blue background and a small turtle icon.

RViz ist ein Tool zur Visualisierung von Informationen aus Topics.

RViz starten:

ros2 run rviz2 rviz2

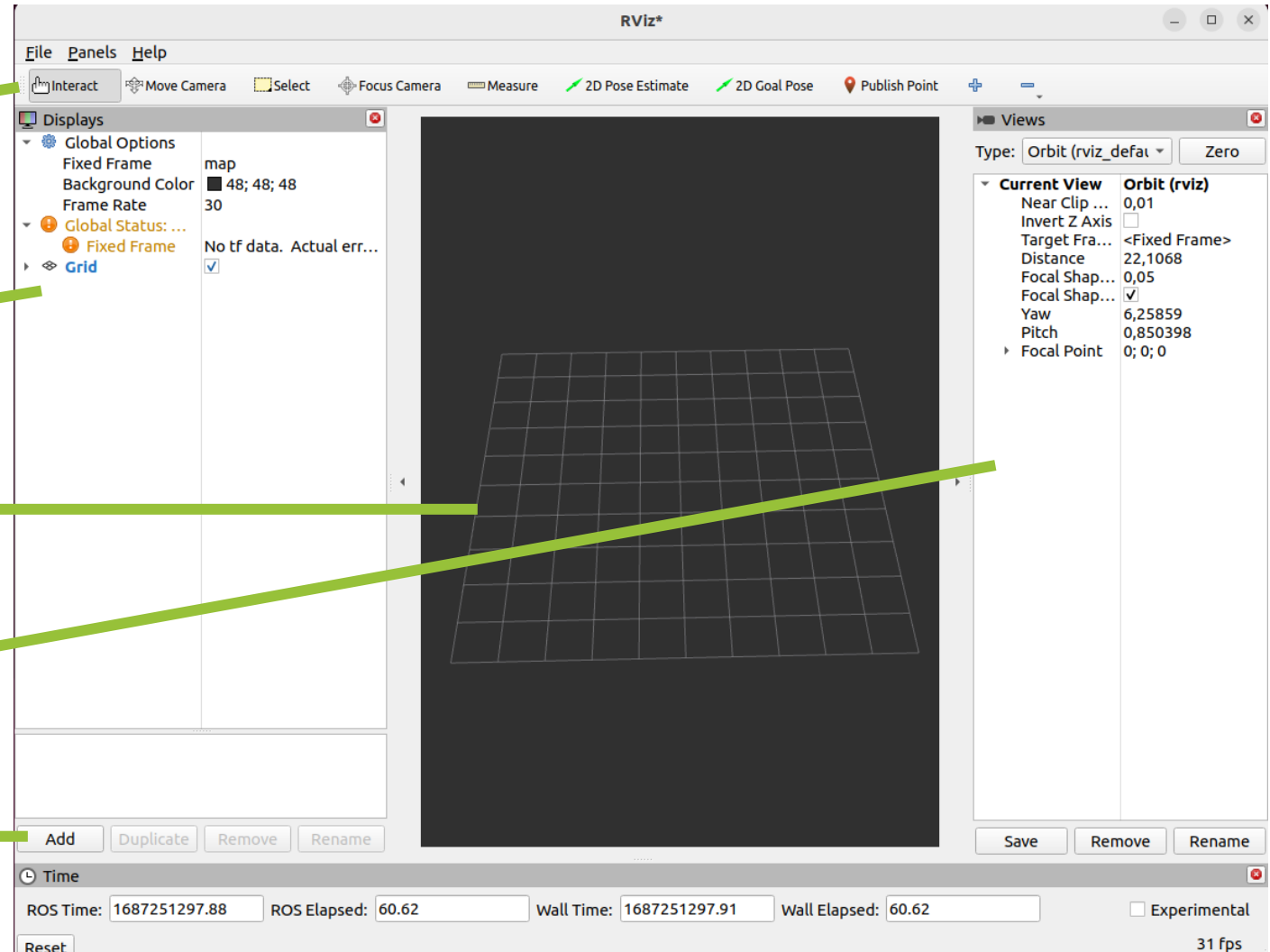
Werkzeuge

Topics sowie Einstellungen und Details

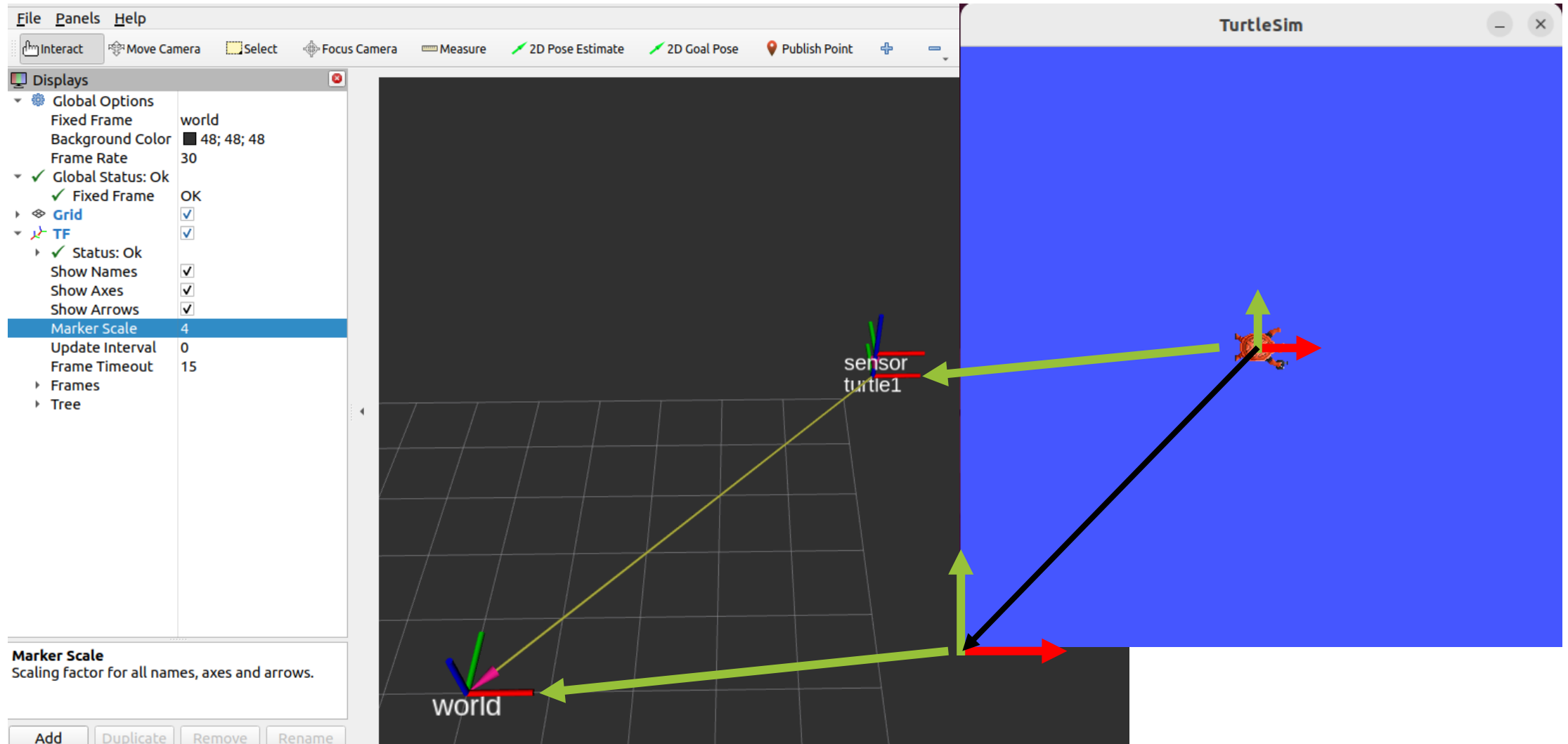
3D-Visualisierung

Kameraeinstellung der 3D-Visualisierung

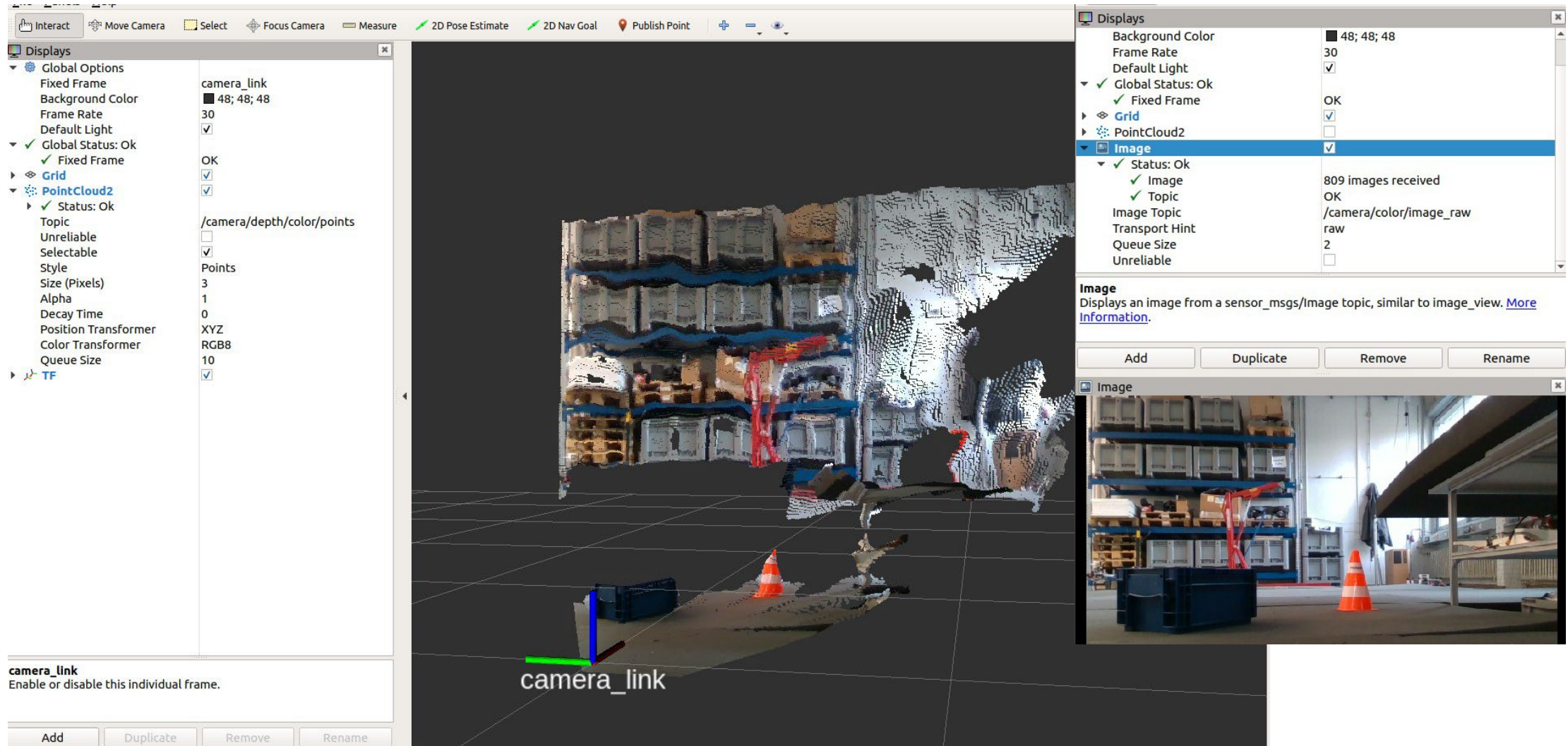
Neue Anzeige / Topics hinzufügen



In RViz können Transformationen der Topics `/tf` und `/tf_static` visualisiert werden.

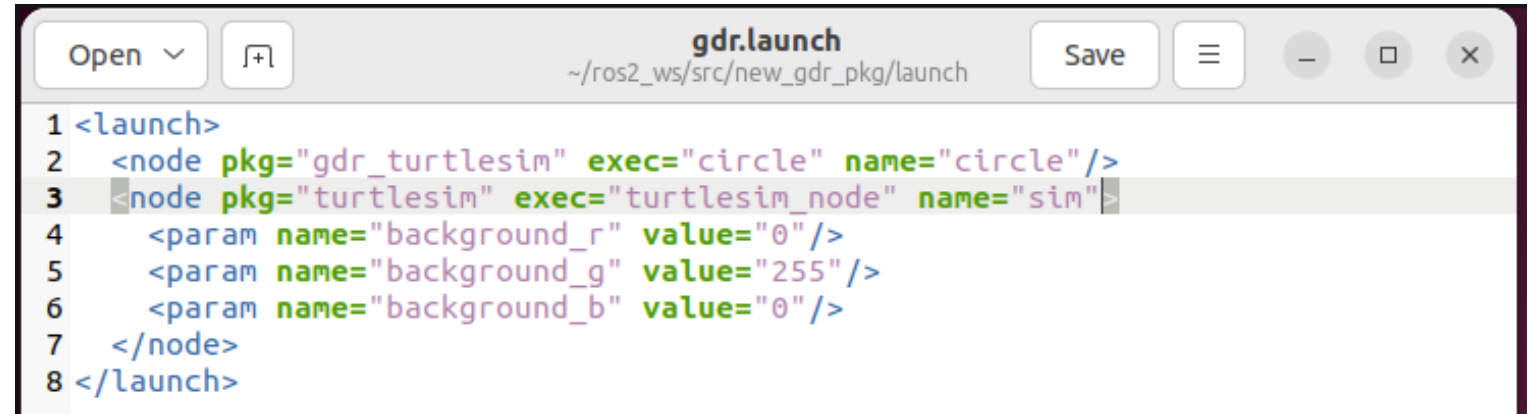


RViz kann viele Topics gleichzeitig visualisieren.



Mit ros2 launch kann das Starten vieler Nodes und das Laden von Einstellungen vereinfacht werden.

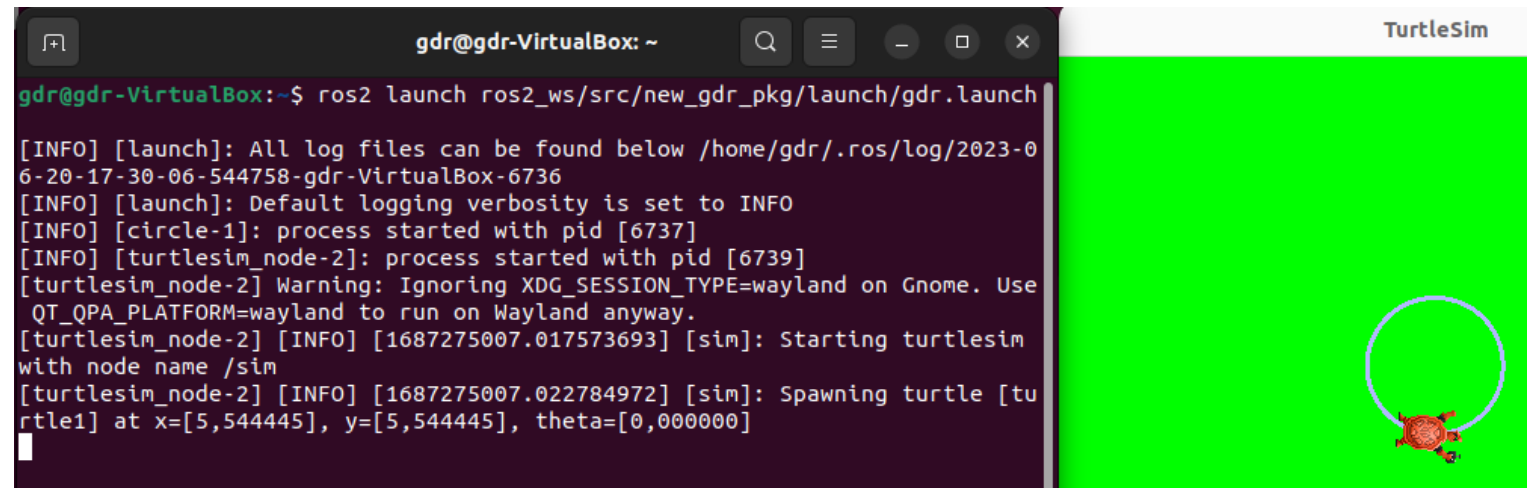
- Viele einzelne Packages, Nodes, Parameter und Einstellungen müssen beim Startprozess bedacht werden
- Einstellung / Start aller benötigten Komponenten mithilfe einer Datei
- Launch-Dateien können in Python, XML und YAML geschrieben werden
- Eine XML-Launch-Datei ist aus Tags aufgebaut:
 - <launch>
 - <node>
 - <include>
 - <param>
 - <arg>
 - ...



```

1 <launch>
2   <node pkg="gdr_turtlesim" exec="circle" name="circle"/>
3   <node pkg="turtlesim" exec="turtlesim_node" name="sim"/>
4     <param name="background_r" value="0"/>
5     <param name="background_g" value="255"/>
6     <param name="background_b" value="0"/>
7 </node>
8 </launch>

```



```

gdr@gdr-VirtualBox: ~
gdr@gdr-VirtualBox:~$ ros2 launch ros2_ws/src/new_gdr_pkg/launch/gdr.launch

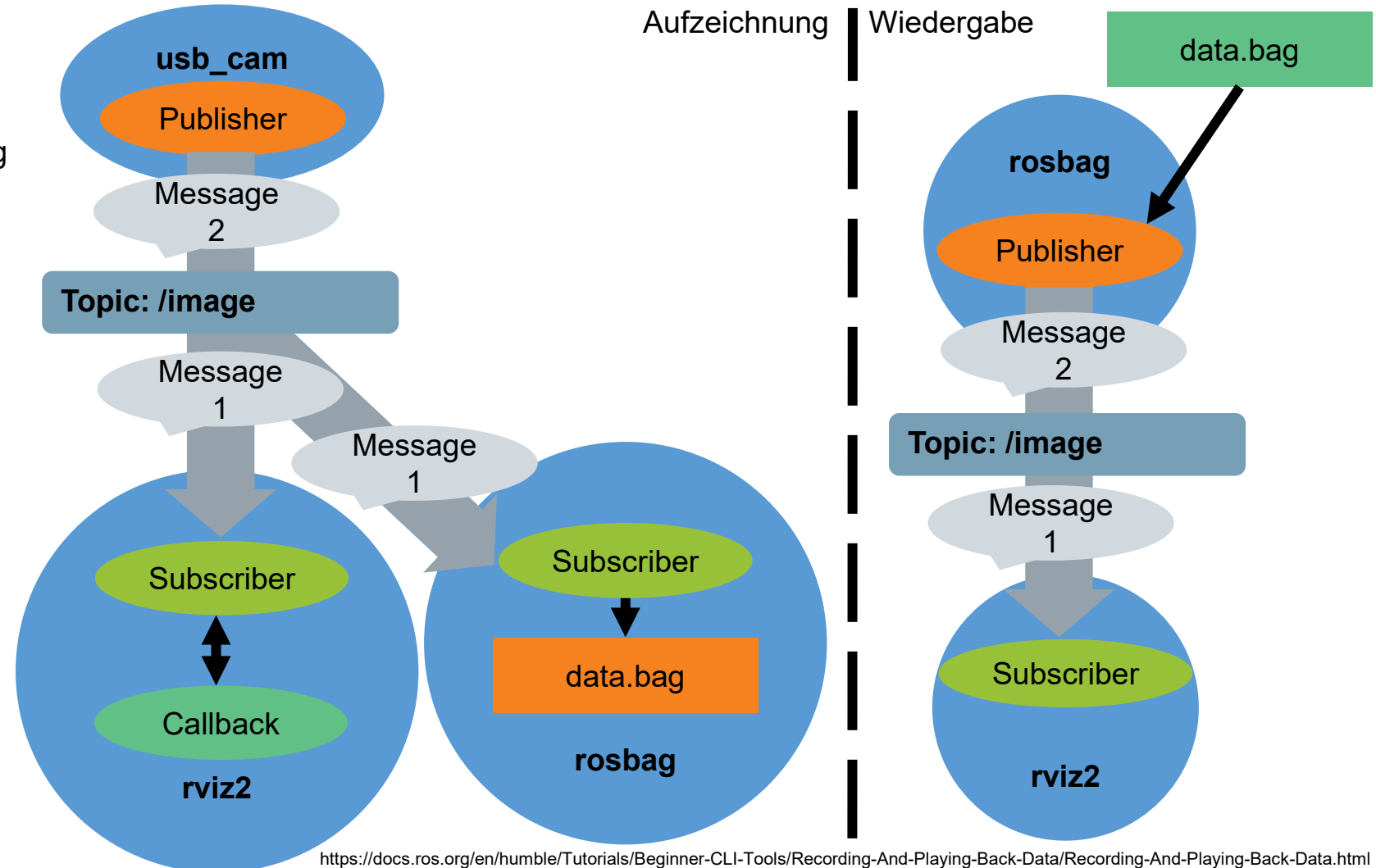
[INFO] [launch]: All log files can be found below /home/gdr/.ros/log/2023-06-20-17-30-06-544758-gdr-VirtualBox-6736
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [circle-1]: process started with pid [6737]
[INFO] [turtlesim_node-2]: process started with pid [6739]
[turtlesim_node-2] Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland to run on Wayland anyway.
[turtlesim_node-2] [INFO] [1687275007.017573693] [sim]: Starting turtlesim with node name /sim
[turtlesim_node-2] [INFO] [1687275007.022784972] [sim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]

```

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Creating-Launch-Files.html>
<https://docs.ros.org/en/humble/How-To-Guides/Launch-file-different-formats.html>

Mit ros2 bag können Topics aufgezeichnet und wiedergegeben werden.

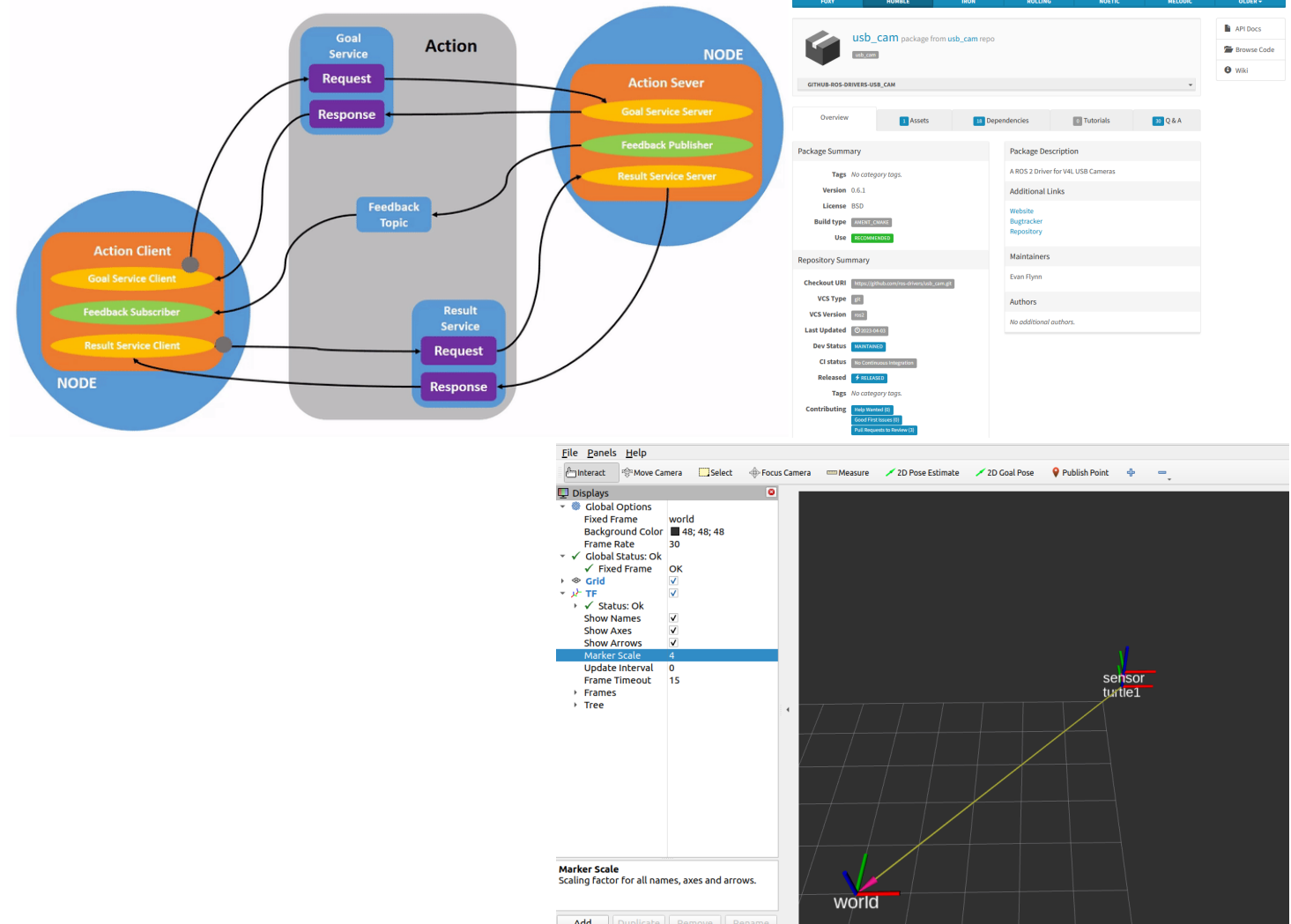
- Topics und deren Messages können in einer .bag Datei aufgezeichnet werden
- Später kann die Aufzeichnung wiedergegeben werden



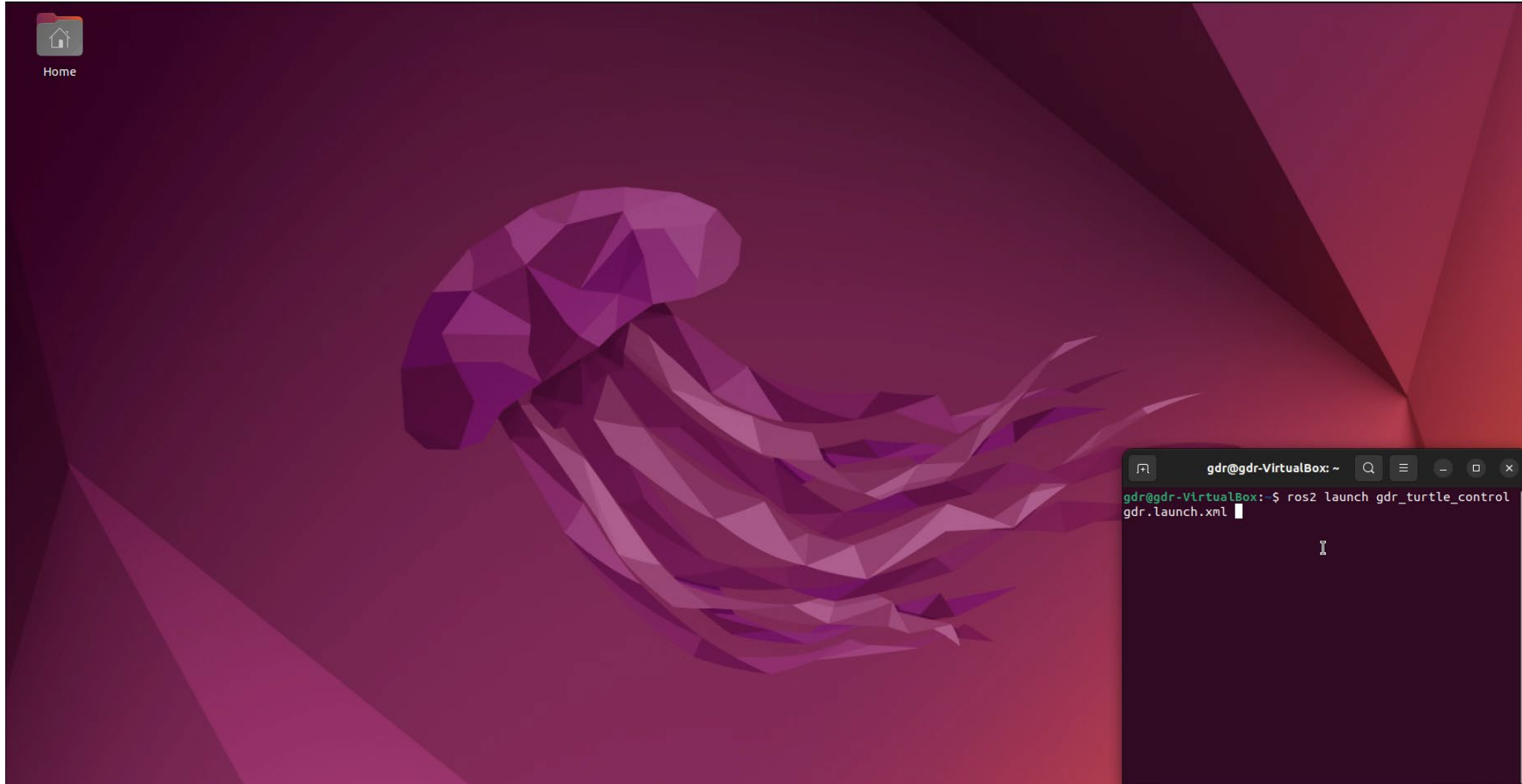
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>

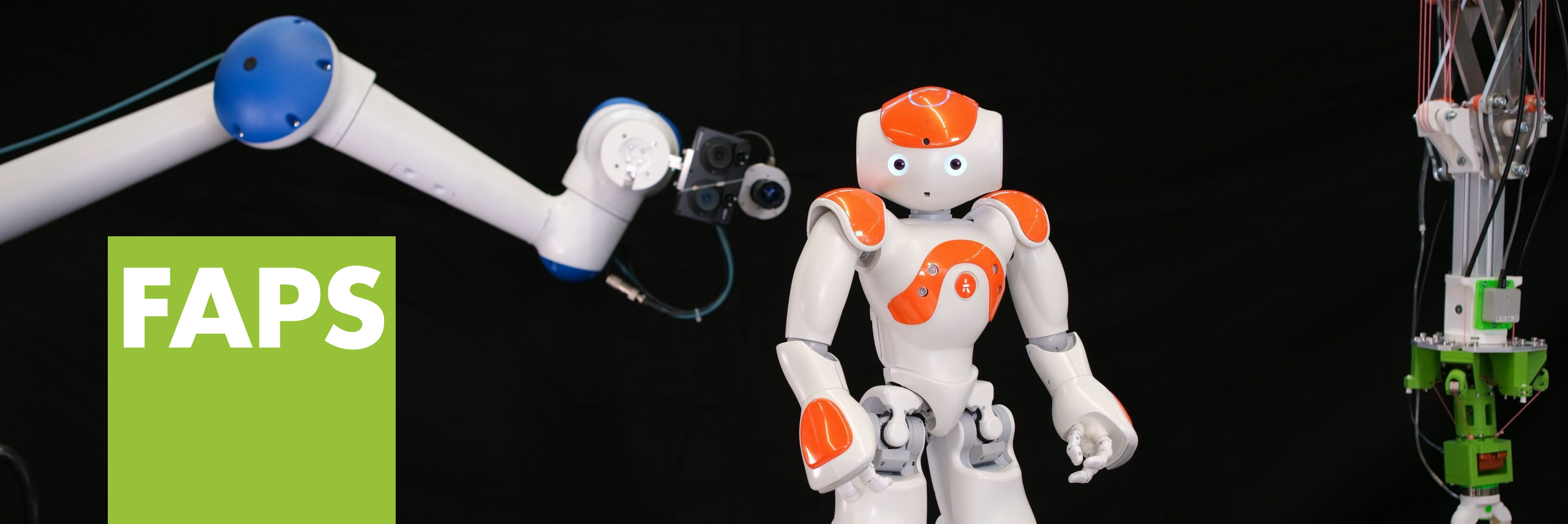
Nach dieser Vorlesungseinheit sollten folgende Fragen beantwortet werden können.

- Was sind ROS Actions und wie funktionieren sie?
- Wie unterscheiden sich die verschiedenen Kommunikationsarten von ROS?
- Wie können Packages der ROS Community genutzt werden und wo findet man Informationen zu den Packages?
- Was ist tf2 und wofür wird es benötigt?
- Welche Tools bietet ROS?



In der Übung zu dieser Vorlesungseinheit wird der Einsatz von RViz zur Steuerung des Turtlesim behandelt.





Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

DANKE